BCSE302P · Database Systems Lab

Project Report

Graph-Based Airline Route Optimization for Disruption Analysis and Sustainable Travel

A Neo4j graph database and the "Neo Airways" web application

NoSQL database: Neo4j 2026.09 Enterprise Edition + Graph Data Science 2026.09

Team member	Register number
Karthick Ganesh	24BCI0018
Amritha Varshini Pakanati	24BCE0390

Course code	Course title	Faculty	Slot
BCSE302P	Database Systems Lab	Lydia Jane G	D1 + TD1

Source code: https://github.com/Karthick-Ganesh/dbms-airline-graph

October 2026

Abstract

Air travellers and planners have to balance journey time against carbon emissions, and a single airport closure caused by fog, storms or system failures can reroute or strand thousands of trips. Answering such questions needs many-hop path queries over a large, highly connected network, which relational databases express only through repeated self-joins.

This project models the global airline network as a labelled property graph in Neo4j. Six public datasets (OurAirports, OpenFlights, the ICAO Aircraft Engine Emissions Databank, an aircraft dataset, recent route data and the 2015 US flight delays dataset) are cleaned and loaded by an idempotent Python pipeline into 12,243 nodes and 61,735 relationships across six labels and eight relationship types, protected by 12 constraints and 18 indexes. Each route edge carries several weights at once (journey time, CO2 per seat and cost), so the fastest and the greenest itinerary come from the same graph.

A FastAPI back end and a responsive web application provide node and relationship CRUD, a route finder that searches every bookable itinerary of up to two stops, GDS algorithms (Dijkstra, A*, Yen's k-shortest paths, Betweenness and PageRank), a disruption simulator that closes airports in one ACID transaction and reports the network-wide impact, a live-traffic map and an interactive graph explorer.

Results show that only 0.43% of city pairs have a nonstop flight and that the CO2 of bookable itineraries for the same trip differs by a median of 46%, so the choice of connection matters. Closing Delhi disrupts 297 routes and 596 daily flights, cuts off 6 airports and adds 1 h 48 m on average to affected trips, whereas Dubai's closure strands no airport. The project shows that a graph database with in-database analytics suits multi-criteria, disruption-aware travel planning.

Keywords: *graph database, Neo4j, Cypher, Graph Data Science, route optimisation, carbon emissions, network resilience, disruption analysis*

Table of Contents

Abstract 2
Part B: Main Report 6
Chapter 1: Introduction 6
1.1 Background of the Study 6
1.2 Motivation 6
1.3 Problem Statement 6
1.4 Aim and Objectives 6
1.5 Scope of the Project 7
1.6 Limitations 7
1.7 Methodology Overview 8
Chapter 2: Literature Review 9
2.1 Overview of the Application Domain and Its Data Needs 9
2.2 Review of Data Models 9
2.3 Review of Existing Systems and Related Work 9
2.4 Comparison of DBMS Platforms and Tools 10
2.5 Standards and Best Practices 10
2.6 Gap Identified 11
2.7 How This Project Addresses the Gap 11
2.8 Summary of the Chapter 11
Chapter 3: System Analysis and Requirements 12
3.1 Existing System and Its Drawbacks 12
3.2 Proposed System and Its Advantages 12
3.3 Feasibility Study 12
3.3.1 Technical Feasibility 12
3.3.2 Economic Feasibility 12
3.3.3 Operational Feasibility 12
3.4 Users, Roles and Access Needs 13
3.5 Requirements Specification 13
3.5.1 Functional Requirements 13
3.5.2 Data Requirements 14
3.5.3 Non-Functional Requirements 14
3.5.4 Hardware Requirements 14
3.5.5 Software Requirements 15
3.6 Business Rules and Constraints 15
Chapter 4: Database Design 16
4.1 Design Approach and Principles 16

4.2 System Architecture Diagram 16
4.3 Data Flow Diagram 17
4.4 Conceptual Design: ER Diagram 18
4.4.1 Entities and Attributes 18
4.4.2 Relationships, Cardinality and Participation 19
4.4.3 Keys 19
4.4.4 Generalization and Specialization 19
4.5 Logical Design 19
4.6 Normalization 23
4.6.1 Functional Dependencies 23
4.6.3 Justified Denormalization 23
4.7 Physical Design 24
4.7.1 Data Types and Storage 24
4.7.2 Indexing Strategy 24
4.7.3 Partitioning 24
4.8 Integrity Constraints 25
Chapter 5: Implementation 26
5.1 Development Environment and Setup 26
5.2 DBMS, Tools and Libraries Used 26
5.5 Views and Indexes Created 27
5.7 Application Connectivity 27
5.8 Version Control and Project Repository 28
5.9 Challenges Faced and Solutions Adopted 29
Chapter 6: Query Design and Performance Tuning 30
6.1 Representative Queries 30
6.1.1 Selection with a spatial predicate 30
6.1.2 Join across four labels 30
6.1.3 Grouping and aggregation 31
6.1.4 Aggregation with a constant applied in Cypher 31
6.1.5 Subquery (COUNT { }) with aggregation 32
6.1.6 Set operation: the bookable itinerary search (UNION ALL) 32
6.1.7 Graph algorithms (GDS) 33
6.1.8 Explicit transaction: closing airports 34
6.2 Performance Tuning 34
Chapter 7: Results and Discussion 36
7.1 Output Screenshots and Sample Reports 36
7.2 Evaluation Against Objectives 42
7.3 Comparison with the Existing System 42

7.4 Discussion of Findings 42
7.5 Limitations Observed 43
Chapter 8: Conclusion and Future Work 44
8.1 Summary of the Work 44
8.2 Key Contributions 44
8.3 Future Enhancements 44
8.4 Concluding Remarks 44
Part C: References and Appendices 45
References 45
Appendix A: Complete DDL Scripts 46
Appendix B: Sample Data and DML Scripts 47
B.1 Sample records 47
B.2 Loading statements (seed scripts) 47
B.3 CRUD statements (crud.py) 49
Appendix C: Stored Procedures, Functions and Triggers (Full Code) 50
C.1 In-database procedures and functions invoked 50
C.2 Transactional functions (stored-procedure equivalents) 50
C.3 Reusable functions 53
C.4 Trigger-equivalent logic 54
Appendix D: Complete Query Set with Outputs 56
Appendix E: Full-Size ER Diagram and Relational Schema 61
Appendix F: Application Screenshots 63
Appendix G: User Manual 71
G.1 Starting the system 71
G.2 Route Finder (Gate 01) 71
G.3 Live Traffic (Gate 02) 71
G.4 Graph Explorer (Gate 03) 71
G.5 Disruption Simulator (Gate 04, administrator) 71
G.6 Analytics (Gate 05) 72
G.7 Manage Data (Gate 06, administrator) 72
G.8 Troubleshooting 72

Part B: Main Report

Chapter 1: Introduction

1.1 Background of the Study

Commercial aviation forms one of the largest transport networks in the world: thousands of airports connected by tens of thousands of scheduled routes, operated by hundreds of airlines with different aircraft. The network is naturally a graph: airports are nodes, routes are directed edges, and airlines, aircraft, flights and disruptions are further entities connected to them. Studies of the worldwide air transportation network have shown that it is highly connected and that a few airports play a disproportionate role in holding it together [10].

Two pressures make this network hard to plan around. First, aviation is a significant and growing source of carbon dioxide: commercial aviation emitted about 918 million tonnes of CO2 in 2018, roughly 2.4% of global CO2 from fossil fuels [11]. Different aircraft, routes and connections can emit very different amounts for the same journey. Second, the network is fragile. Weather, fog and air-traffic-control failures close airports and corridors at short notice; during Winter Storm Juno (26-28 January 2015) the real flight data used in this project contains 1,903 weather cancellations at ten US airports, with up to 97% of one airport's departures cancelled in a day.

1.2 Motivation

Popular flight search tools optimise price and journey time, and some now show emission estimates, but they do not let a user explore the trade-off between time and CO2 across connections, nor ask "what happens if this airport closes?". Airline and airport operations centres do run such analyses, but with proprietary tools. A course project on database systems is a good opportunity to show that the required questions (multi-hop paths, weighted shortest paths, centrality, spatial proximity and what-if simulations) can be answered directly inside a database, using freely available data.

These questions are awkward in a relational database: "all airports reachable with at most one stop" is a self-join of the routes table, each extra stop adds another join, and weighted shortest paths need procedural code outside SQL. A native graph database stores relationships as first-class records and traverses them without joins, which motivated the choice of Neo4j.

1.3 Problem Statement

Design and implement a graph database of the global airline network, built from public datasets, that (a) finds realistic, bookable itineraries that are fastest, greenest or shortest between any two airports, (b) simulates the closure of airports or whole regions reversibly and measures the resulting damage to the network, and (c) supports the full set of database operations (CRUD, integrity constraints, indexes, transactions and analytical queries) through an easy-to-use, responsive web application.

1.4 Aim and Objectives

Aim: to build a disruption-aware, carbon-aware airline route optimisation system on a native graph database.

Table 1: Project objectives

ID	Objective
O1	Design a property-graph model of airports, airlines, aircraft, engines, flights and disruption events from an ER model, with multi-criteria weights on route edges.
O2	Build a reproducible, idempotent pipeline that cleans, validates and loads six public datasets into Neo4j.
O3	Implement node and relationship CRUD with constraints and indexes that protect integrity and support the queries.
O4	Find the fastest, greenest and shortest bookable itineraries (at most two stops, bounded extra time), and expose the CO2 differences between choices.
O5	Simulate airport and regional closures with ACID transactions and report their network-wide impact; make closures fully reversible.
O6	Provide network analytics: graph algorithms (centrality, k-shortest paths, A*), aggregations, temporal and spatial queries.
O7	Deliver a responsive web application with an interactive graph visualisation, maps and a simulated live-traffic view.

1.5 Scope of the Project

- Global scope: 9,120 airports, 1,123 airlines, 106 aircraft types and 888 engine records; 56,627 directed routes between 3,635 airports with scheduled service.
- Flight-level data: 1,000 flights from January 2015 (600 real US flights from the US Bureau of Transportation Statistics [15] and 400 synthetic flights for India and other regions sampled from the real data).
- Route optimisation on time, CO2 per seat and distance; disruption simulation of airports and regions; analytics; a simulated live-traffic view built from the route graph.
- Single-user, single-machine deployment on Neo4j Desktop; a guest role for viewing and an administrator role for changes.
- Out of scope: ticket prices and booking, live passenger logistics, live air-traffic-control or collision-avoidance integration, and live flight positions.

1.6 Limitations

- Aircraft fuel burn per kilometre is synthetic: a seeded draw of a realistic per-seat efficiency for each aircraft category, multiplied by typical seats. Real burn also depends on stage length, load and weather.
- Flight records cover January 2015, while routes come from recent data; the flight layer is used for delay and disruption analysis, not for current schedules.
- Journey time assumes a fixed 90-minute connection at every stop; real minimum connection times vary by airport.
- Live traffic is simulated (one daily departure per route and airline at a fixed local time); real-time position feeds are commercial or rate-limited.
- The web pages load map and graph libraries and map tiles from the internet.

1.7 Methodology Overview

The project followed the database life cycle. Each phase produced a concrete artefact, and later phases fed corrections back into earlier ones (for example, the carbon model was revised after testing showed unrealistic results).

Table 2: Database life cycle followed

Phase	Activities	Artefacts
Requirements	Rubric analysis; functional and data requirements; user roles	Chapter 3; `bcse302p_rubrics.md`
Conceptual design	Entities, attributes, relationships and cardinalities	ER diagram (Figure 3)
Logical design	Mapping the ER model to a property graph: labels, relationship types, properties, keys	Graph schema (Figure 4), Section 4.5, design document
Physical design	Data types, constraints, indexes, memory settings, GDS projection	Sections 4.7-4.8, `seed5.py`
Implementation	Data pipeline, CRUD module, queries, REST API, web application	`seed*.py`, `crud.py`, `queries.py`, `traffic.py`, `app.py`, `static/`
Testing	Dry runs of every loader; a 57-check end-to-end API test; browser testing at phone, tablet and desktop sizes; PROFILE of key queries	Chapter 6, Section 7.2
Maintenance	Idempotent re-seeding, soft-state disruptions, version control, documented backup procedure	Git history, README, Section 3.5.3

Chapter 2: Literature Review

2.1 Overview of the Application Domain and Its Data Needs

An airline network application needs reference data about airports (identity, location, time zone, capacity), airlines, aircraft types and their engines, and operational data about routes and individual flights (schedules, delays, cancellations) and about disruptions. Its characteristic queries are relationship-centric: which airports are reachable, which path is fastest or emits least, which airports are critical, and what happens when part of the network is removed. The data also has a spatial dimension (airport coordinates, great-circle distances) and a temporal one (time-zoned schedules and event windows).

2.2 Review of Data Models

The relational model [1] stores data in normalised tables and expresses relationships through foreign keys and joins; it is mature and strongly consistent, but recursive path queries need recursive common table expressions or procedural code [19], [20]. NoSQL models relax the fixed schema in different ways, and NewSQL systems keep SQL and ACID while scaling horizontally [4]. Graph databases store relationships as first-class records, so traversals follow pointers instead of computing joins [2], [3].

Table 3: Comparison of data models

Model	Typical systems	Strengths	Fit for this project
Relational	PostgreSQL, MySQL, Oracle	Mature, ACID, SQL, strong integrity	Multi-hop and weighted path queries need many self-joins or extensions (e.g. pgRouting)
Object-relational	PostgreSQL types, Oracle objects	Complex types inside tables	Same join problem for paths
Document	MongoDB, Couchbase	Flexible nested documents	Relationships across documents need application-side joins or $graphLookup
Key-value	Redis, DynamoDB	Very fast lookups by key	No relationship queries
Wide-column	Cassandra, HBase	Massive write throughput, time series	Query-first design; no traversals
Graph (property graph)	Neo4j, Amazon Neptune, TigerGraph	Index-free traversal, path queries, graph algorithms, relationship properties	Chosen: routes are relationships; paths, centrality and what-if analysis are native
NewSQL	CockroachDB, Google Spanner	SQL + ACID + horizontal scale	Scale is not the problem here; path queries remain joins

Reasons for choosing the property-graph model: (1) routes, hubs, flights and disruptions are relationships with their own properties (distance, weights, status); (2) variable-length patterns such as `(a)-[:ROUTE*1..2]->(b)` express multi-stop reachability in one line; (3) the Graph Data Science library runs Dijkstra, A*, Yen's, Betweenness and PageRank inside the database; (4) the schema is flexible, which suited combining six heterogeneous datasets.

2.3 Review of Existing Systems and Related Work

- **Flight search engines (Google Flights, Skyscanner).** Optimise price and duration; Google Flights shows emission estimates based on its open Travel Impact Model [24]. They do not let

users bound the number of stops and extra time explicitly when looking for the greenest option, and offer no disruption analysis.

- **Flight tracking (Flightradar24, OpenSky Network).** Show live aircraft positions from ADS-B receivers. Commercial APIs are paid; the OpenSky Network offers research access with rate limits [26], [27].
- **ICAO Engine Emissions Databank.** Certified engine data such as landing/take-off fuel, used here to charge every leg for its take-off and landing [12].
- **OpenFlights and OurAirports.** Open datasets of airports, airlines and routes that are widely used in research and teaching [13], [14].
- **Network science studies.** Guimerà et al. showed that the worldwide air network has "anomalous centrality": some airports with modest traffic are among the most central because they bridge regions [10]. Our Betweenness results reproduce this (Anchorage ranks second).
- **Graph algorithms.** Dijkstra's shortest-path algorithm [5], A* search [6], Yen's k-shortest loopless paths [7], Brandes' betweenness algorithm [8] and PageRank [9] are the standard methods, all available in Neo4j GDS [17].

2.4 Comparison of DBMS Platforms and Tools

Table 4: Comparison of candidate DBMS platforms

Platform	Query language	Graph algorithms	Local development	Notes
Neo4j 2026 (chosen)	Cypher (GQL-aligned)	GDS library: 60+ algorithms in memory	Neo4j Desktop, free developer licence	Mature tooling (Browser, drivers); Enterprise adds existence and type constraints
Amazon Neptune	Gremlin, openCypher, SPARQL	Neptune Analytics (separate service)	Cloud only	Managed, paid
TigerGraph	GSQL	Built-in library	Docker / cloud	Strong at very large graphs; less common in teaching
ArangoDB	AQL (multi-model)	Pregel-style, limited	Local	Document + graph
PostgreSQL + pgRouting	SQL	Shortest paths only	Local	Relational joins for traversal; no centrality
MongoDB	MQL	$graphLookup (recursive lookup)	Local	No weighted shortest paths

2.5 Standards and Best Practices

- **Query language.** Cypher [16] follows the ISO/IEC 39075 GQL standard for graph query languages [18]; all queries are parameterised, which prevents injection.
- **Data standards.** IATA airport and airline codes [25] and ICAO aircraft type codes as keys; WGS-84 coordinates (SRID 4326) stored as native points; ISO 8601 date-times with UTC offsets.
- **Transactions.** Each CRUD call is one managed ACID transaction; multi-step changes (closures) use one explicit transaction that commits or rolls back as a whole.
- **Data engineering.** Idempotent loads (`MERGE` on unique keys), constraints created before data, seeded randomness for reproducibility, dry-run validation, and provenance properties (`data_source`, `fuel_burn_source`).
- **Security.** Secrets kept in a git-ignored `.env` file, constant-time password comparison, signed session cookies, and writes restricted to the administrator.

- **Engineering.** Version control on GitHub, documented setup, and a design document kept in sync with the code.

2.6 Gap Identified

- No freely available tool combines carbon-aware routing with realistic booking constraints (stops and extra time) and shows how much the choice of connection matters.
- Disruption what-if analysis with a measured network-wide impact (stranded airports, delayed trips, lost seats) is limited to proprietary airline and airport tools.
- Course projects on databases rarely integrate real engine emission data, real delay data and graph algorithms in one reproducible system.

2.7 How This Project Addresses the Gap

- Multi-weight `ROUTE` edges (time, CO2 per seat, cost) and an exact Cypher search over all itineraries of up to two stops, bounded by an extra-time rule, with the CO2 range of all bookable choices shown.
- Soft-state closures applied in one ACID transaction, a single-source Dijkstra before/after comparison that quantifies the impact, and exact restoration.
- A reproducible pipeline from six public datasets, ICAO engine data linked to aircraft through `POWERED_BY`, and analytics in the database through GDS.

2.8 Summary of the Chapter

The literature and existing systems show that the airline network is best treated as a graph, that the choice of route and aircraft strongly affects emissions, and that network resilience depends on a few critical airports. Among the data models reviewed, the property graph fits the workload best, and Neo4j with its Graph Data Science library provides the query language, algorithms and tooling needed. The next chapter turns these findings into requirements.

Chapter 3: System Analysis and Requirements

3.1 Existing System and Its Drawbacks

- Flight search sites focus on price and duration; emission figures are shown per result, but the user cannot ask for the greenest itinerary within a chosen number of stops and extra hours.
- No public tool answers "what if this airport closes?" with a measured impact.
- Relational storage of route data turns multi-stop questions into chains of self-joins and moves path algorithms into application code.
- Live tracking data is commercial; open datasets are scattered, inconsistent and need significant cleaning before use.

3.2 Proposed System and Its Advantages

- A single graph database containing all reference, route, flight and disruption data, with relationships stored natively.
- Fastest, greenest and shortest bookable itineraries from the same graph, with the CO2 spread of all choices.
- Reversible disruption simulation in one transaction, with an impact report and an audit trail of events.
- Network analytics and visualisation (graph explorer, maps, live-traffic view) in a web application usable on phones and desktops.
- Reproducible: the whole database can be rebuilt from public data with five scripts.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All components are available and run on a 16 GB laptop: Neo4j 2026.09 Enterprise through Neo4j Desktop, the GDS plugin, Python 3.10 and standard libraries. The graph (12,243 nodes, 61,735 relationships, about 220 MB on disk) fits comfortably in memory; the in-memory GDS projection of the active network has 3,670 nodes and 56,627 relationships. Measured response times (Chapter 6) are well under a second for searches and analytics.

3.3.2 Economic Feasibility

The project has no direct cost: Neo4j Desktop includes a free developer licence for the Enterprise edition, GDS is free for single-machine use, and all datasets, libraries and map tiles are open. Development used existing laptops.

3.3.3 Operational Feasibility

Users need only a web browser. The interface uses familiar boarding-pass styling, airport search by city or code, and plain-language results. Administrators manage data through forms instead of Cypher. Operating the system needs one command (`uvicorn app:app`) with Neo4j running.

3.4 Users, Roles and Access Needs

Table 5: Users and roles

Role	Who	Access
Guest (traveller, analyst)	Anyone opening the web application	Read only: route finder, live traffic, graph explorer, analytics, past disruption events
Administrator	Logged in with the credentials in `.env`	Everything a guest can do, plus create/update/delete of airports, airlines, flights, routes, hubs and events, and disruption simulation and restore
Database administrator / developer	Project team	Neo4j Desktop and Browser, seed scripts, configuration (heap, page cache), backups

3.5 Requirements Specification

3.5.1 Functional Requirements

Table 6: Functional requirements

ID	Requirement
FR1	Load and clean airports, airlines, aircraft, engines, routes, flights and disruption events from the public datasets; re-running the load must not duplicate data.
FR2	Create, read, update and delete airports, airlines, flights, routes, hub links and events; airports are closed (soft delete) rather than deleted.
FR3	Search airports by name, city or code (full-text).
FR4	Find the fastest, greenest and shortest itineraries with at most N stops (N = 0, 1, 2) and a bounded extra time; list alternatives and the CO2 range of all bookable itineraries.
FR5	Show the unconstrained optimum from a graph algorithm for comparison.
FR6	Close one or more airports or a whole region in one transaction, log a disruption event, and report the impact; restore exactly what was closed.
FR7	Run analytics: betweenness, PageRank, A*, Yen's k-shortest paths, on-time performance, event impact, spatial search, carbon efficiency, engine comparison and hub dependency.
FR8	Explore the graph interactively (airport, flight and event neighbourhoods) as a node-link graph and on a map.
FR9	Show simulated live traffic for the current time and replay the real January 2015 flights.
FR10	Restrict all changes to an authenticated administrator.

3.5.2 Data Requirements

Table 7: Data requirements

Entity	Source	Volume	Expected growth
Airport	OurAirports `airports_new.csv`, `runways.csv`	9,120 nodes (of 61,221 rows)	Slow (new airports are rare)
Airline	OpenFlights `airlines.csv` + route data	1,123	Slow
Aircraft	Commercial aircraft dataset	106 types	A few new types per year
Engine	ICAO Engine Emissions Databank v32	888	With each databank release
ROUTE	Route JSON (distance, duration, carriers) + OpenFlights routes and planes (aircraft)	56,627 edges	Seasonal changes; edges updated in place
Flight	Kaggle / BTS 2015 + synthetic	1,000 (sample of 469,968 January rows)	Highest: ~100,000 flights per day worldwide in a production system
DisruptionEvent	Detected from cancellation spikes; created in the app	6	Grows with every simulated or real event

3.5.3 Non-Functional Requirements

Table 8: Non-functional requirements

Category	Requirement and how it is met
Performance	Route search under 1 s (measured 0.14-0.41 s); analytical queries under 0.5 s (8-293 ms); live-traffic list under 0.3 s once the timetable is built; disruption impact report in about 3 s (up to about 9 s on a cold start).
Concurrency	Each request runs in its own managed transaction; FastAPI runs requests in a thread pool; a lock prevents the shared GDS projection from being rebuilt while it is being queried.
Availability	The application starts even if Neo4j is down and reports a clear 503 error; the GDS projection and the traffic timetable are rebuilt lazily after changes.
Security	Admin-only writes; password from `.env` compared in constant time; signed, same-site session cookie; parameterised Cypher (no injection); input validation with Pydantic.
Backup and recovery	Neo4j's write-ahead transaction log recovers committed transactions after a crash. The database can be rebuilt from the public data with the seed scripts, and closures are reversible. A backup procedure with `neo4j-admin database dump` is documented (Section 4.2) but not automated.
Usability	Responsive layout from 375 px phones to desktops; airport autocomplete; plain-language summaries.

3.5.4 Hardware Requirements

Table 9: Hardware requirements

Component	Minimum	Used for development
Processor	64-bit dual core	64-bit multi-core laptop
Memory	8 GB RAM	16 GB RAM (Neo4j: 4 GB heap, 1 GB page cache)
Storage	2 GB free (database ~220 MB; flights dataset 590 MB)	SSD
Network	Needed once for downloads, and for map tiles in the browser	Broadband

3.5.5 Software Requirements

Table 10: Software requirements

Software	Version	Purpose
Neo4j Desktop / Neo4j Enterprise	2026.09.0	Graph DBMS
Graph Data Science plugin	2026.09.0	Graph algorithms
Python	3.10	Data pipeline and back end
neo4j (Python driver)	6.x	Bolt connectivity
pandas, openpyxl, tzdata	current	Data cleaning, Excel reading, time zones
FastAPI, Uvicorn, itsdangerous	0.161, 0.54, current	REST API, ASGI server, signed cookies
Leaflet, vis-network	1.9.4, 9.1.9	Maps and graph visualisation in the browser
Any modern browser	Chrome, Edge, Firefox	User interface
Git and GitHub	current	Version control

3.6 Business Rules and Constraints

Table 11: Business rules

ID	Rule	Enforced by
BR1	Every airport, airline, aircraft type, engine, flight and event has a unique key (IATA/ICAO code or generated id).	Unique constraints
BR2	An airport must have a name and a point location; a flight must have a status and a zoned departure time.	Existence and type constraints
BR3	Route weights `weight_time` and `weight_carbon` are floating-point numbers.	Relationship property type constraints
BR4	A flight departing 15 or more minutes late is DELAYED; the delay is derived from actual minus scheduled departure.	`crud.record_departure`
BR5	Airports and corridors are never deleted to model a disruption; their status changes (ACTIVE / RESTRICTED / CLOSED, ACTIVE / DISRUPTED / CLOSED). Only ACTIVE parts are used for routing.	Soft state + filters in queries and projection
BR6	A bookable itinerary has at most two stops; a greener or shorter option may take at most 75% longer than the fastest (50% if the fastest takes 15 h or more), and never more than 10 h longer.	`queries.find_routes`, `extra_time_allowed`
BR7	Each connection adds 90 minutes to the journey time.	`CONNECTION_MIN`, `weight_trip`
BR8	CO2 = fuel burned x 3.16 kg per kg of jet fuel; per-seat CO2 uses typical seats.	`seed3.py`, Query 10
BR9	An airport that still has routes cannot be hard-deleted; only events created manually can be deleted.	API checks (HTTP 409)
BR10	Only the administrator can change data or simulate disruptions.	`require_admin` dependency

Chapter 4: Database Design

4.1 Design Approach and Principles

- **Query-driven modelling.** The model was shaped by the questions to answer: paths between airports, weights on those paths, and the effect of closures.
- **Two layers.** `ROUTE` edges between airports hold precomputed aggregate weights for pathfinding; `Flight` nodes (a reified relationship) hold individual schedules and delays. Graph algorithms run on the route layer only.
- **Multiple weights on one edge.** Each route stores time, CO2 per seat and cost, so different optimisations use the same graph by changing only the weight property.
- **Soft state.** Disruptions change status properties instead of deleting data, which keeps closures reversible and auditable.
- **Provenance.** Every derived or synthetic value records where it came from (`data_source`, `fuel_burn_source`, `duration_estimated`).

4.2 System Architecture Diagram

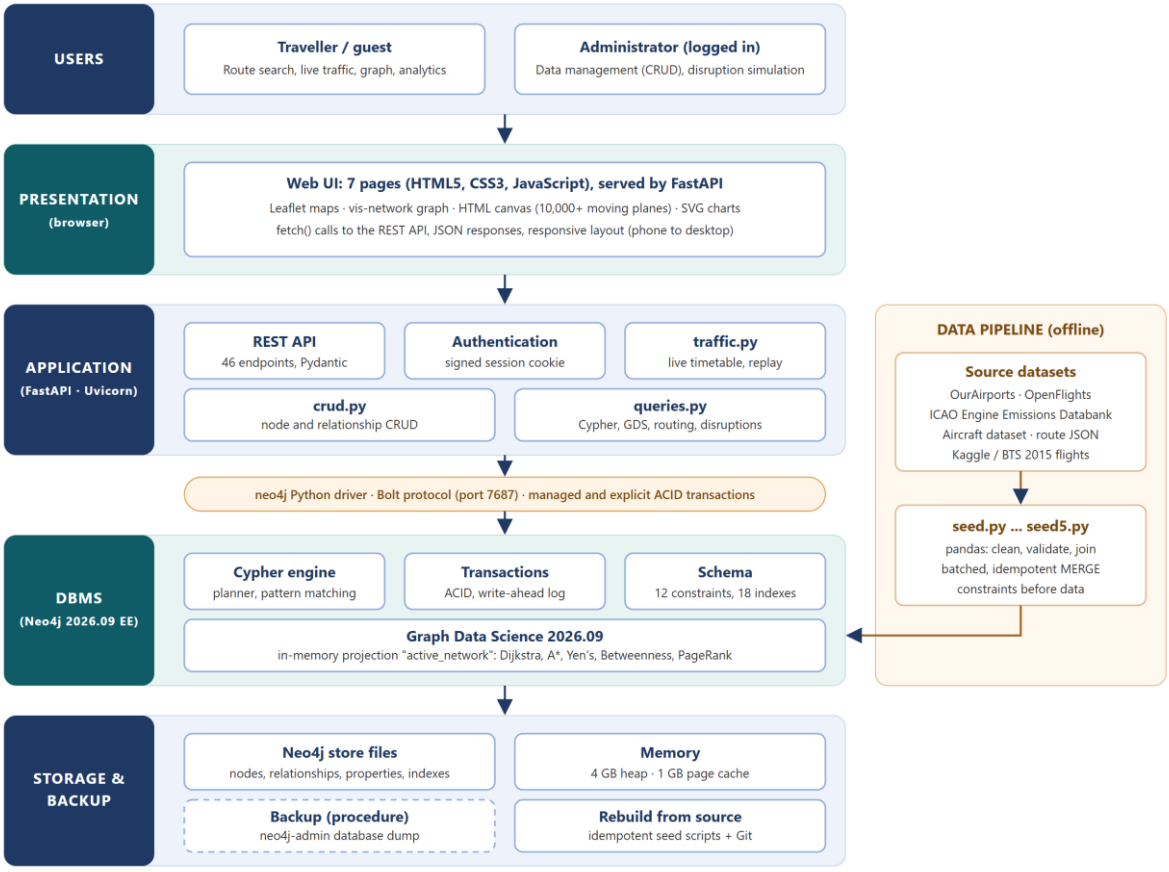


Figure 1: Layered system architecture

Table 12: Architecture components

Layer	Components and responsibilities
Users	Guests explore routes, traffic and analytics; the administrator also edits data and simulates disruptions.
Presentation	Seven HTML pages (route finder, live traffic, graph explorer, disruptions, analytics, data management, login) with JavaScript modules. Leaflet draws maps, vis-network draws the graph, an HTML canvas draws thousands of moving aircraft, and SVG draws charts.
Application	FastAPI on Uvicorn: 46 REST endpoints with Pydantic validation, session authentication, and three modules: `crud.py` (CRUD), `queries.py` (Cypher, GDS, routing, disruptions) and `traffic.py` (live timetable and replay).
Connectivity	The official neo4j Python driver over the Bolt protocol (port 7687), using managed transactions for single statements and explicit transactions for multi-step changes.
DBMS	Neo4j 2026.09 Enterprise: Cypher planner and runtime, ACID transactions with a write-ahead log, 12 constraints and 18 indexes, and the GDS library with the in-memory projection `active_network`.
Storage and backup	Native store files for nodes, relationships and properties; 4 GB heap and 1 GB page cache. Recovery: transaction log replay; rebuild from source data with the idempotent seed scripts; backup procedure `neo4j-admin database dump neo4j --to-path=backups/` with the database stopped.
Data pipeline	Offline Python scripts (`seed.py` to `seed5.py`) that read the datasets with pandas, clean and join them, and load them with batched `MERGE` statements.

4.3 Data Flow Diagram

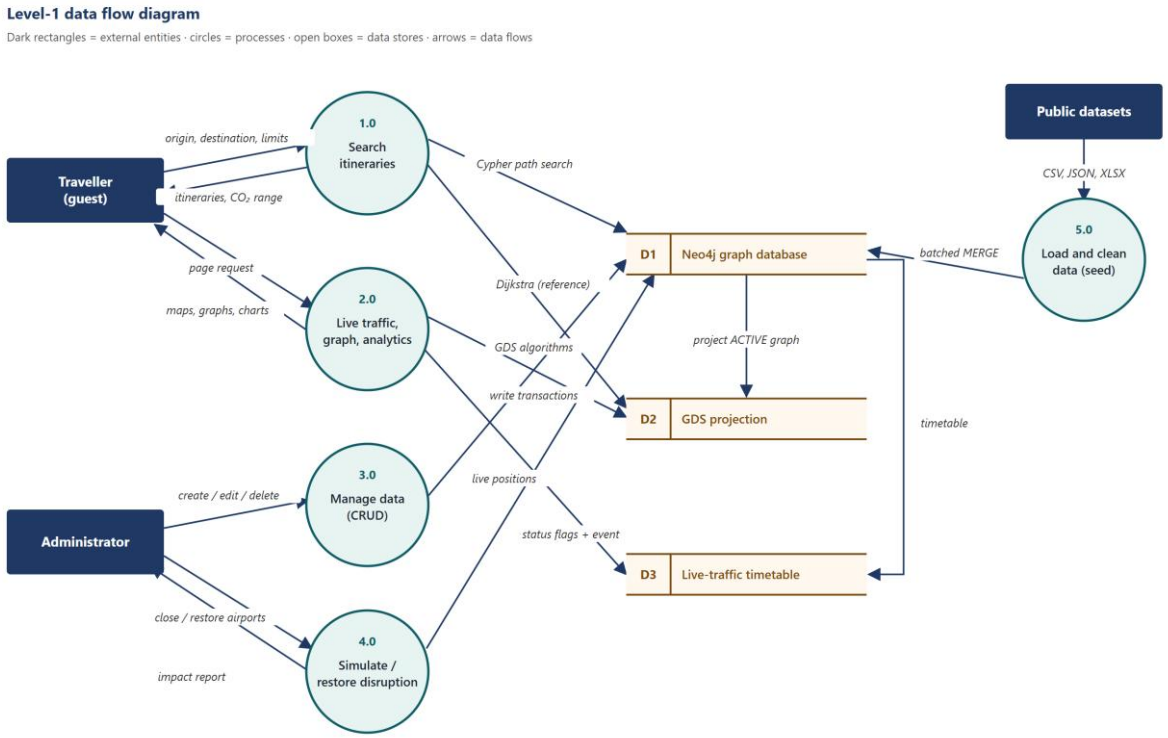


Figure 2: Level-1 data flow diagram

Travellers send an origin, destination and limits to process 1.0, which reads the graph (D1) through a Cypher path search and the GDS projection (D2) for the reference optimum. Process 2.0 serves the live-traffic view from a timetable cache (D3) and the analytics from D2. Administrators change data

through process 3.0 and simulate closures through process 4.0, which writes status flags and an event in one transaction and returns an impact report. Process 5.0 loads the public datasets into D1.

4.4 Conceptual Design: ER Diagram

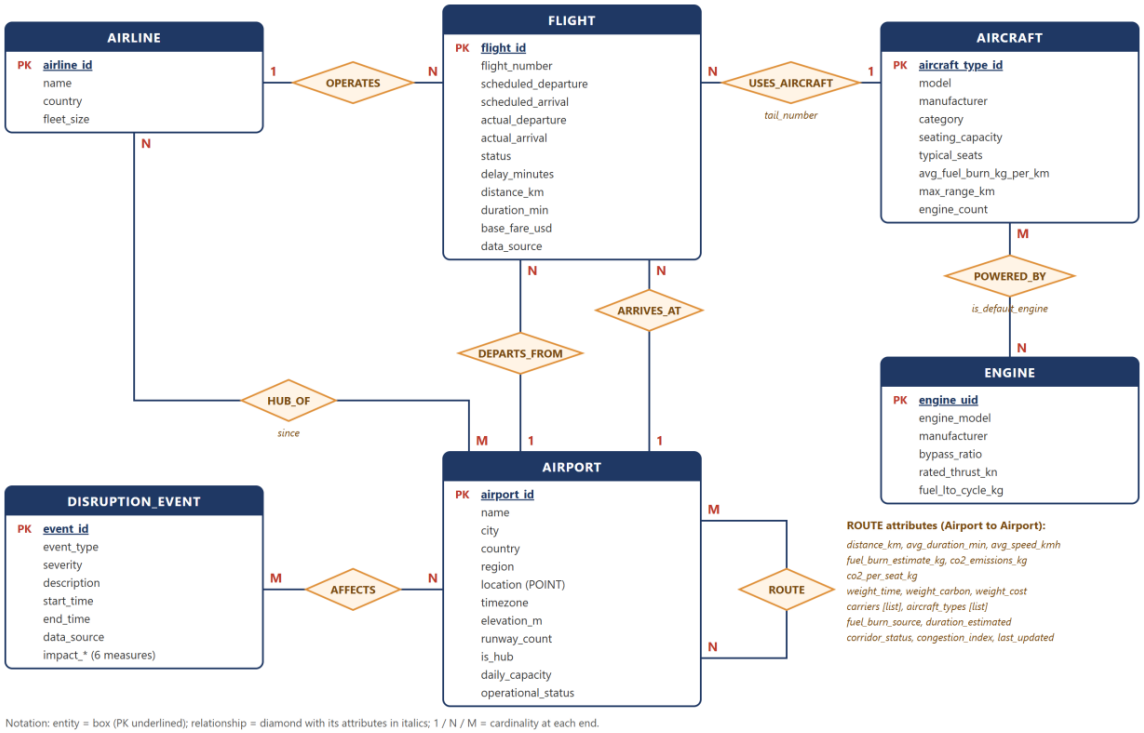


Figure 3: Entity-relationship diagram

4.4.1 Entities and Attributes

Table 13: Entities

Entity	Description	Key attributes
AIRPORT	An airport with scheduled or potential service	airport_id (PK), name, city, country, region, location, timezone, operational_status
AIRLINE	An operating carrier	airline_id (PK), name, country, fleet_size
AIRCRAFT	An aircraft type (not an individual airframe)	aircraft_type_id (PK), model, category, seating_capacity, typical_seats, avg_fuel_burn_kg_per_km
ENGINE	A certified engine model from the ICAO databank	engine_uid (PK), engine_model, bypass_ratio, rated_thrust_kn, fuel_lto_cycle_kg
FLIGHT	One scheduled flight on one day	flight_id (PK), flight_number, scheduled/actual times, status, delay_minutes, data_source
DISRUPTION_EVENT	A real or simulated disruption, kept as an audit record	event_id (PK), event_type, severity, start_time, end_time, impact measures

4.4.2 Relationships, Cardinality and Participation

Table 14: Relationships

Relationship	Between	Cardinality	Participation	Attributes
OPERATES	Airline -> Flight	1 : N	Every flight has exactly one airline (total); an airline may have none	-
DEPARTS_FROM	Flight -> Airport	N : 1	Total for Flight	-
ARRIVES_AT	Flight -> Airport	N : 1	Total for Flight	-
USES_AIRCRAFT	Flight -> Aircraft	N : 1	Total for Flight	tail_number
ROUTE	Airport -> Airport	M : N (directed)	Partial: 3,635 of 9,120 airports have routes	distance, durations, fuel, CO2, three weights, carriers, aircraft types, status
HUB_OF	Airport -> Airline	M : N	Partial	since (not available in the data)
POWERED_BY	Aircraft -> Engine	M : N	Partial: 81 of 106 types (turboprops are not in the databank)	is_default_engine
AFFECTS	DisruptionEvent -> Airport	M : N	Total for DisruptionEvent	-

4.4.3 Keys

Natural keys from aviation standards are used where they exist: IATA codes for airports (`airport_id`) and airlines (`airline_id`), ICAO type designators for aircraft (`aircraft_type_id`) and the databank's unique id for engines (`engine_uid`). Flights use a composite natural key `{carrier}{number}-{yyyymmdd}` (for example `B61403-20150127`), which is unique because only one leg per flight number and day is kept. Events use generated ids such as `EVT-2015-0126-WEATHER`. In the graph, relationships replace foreign keys: a relationship always connects two existing nodes.

4.4.4 Generalization and Specialization

Not used. Aircraft categories (widebody, narrowbody, regional, turboprop, private jet, freighter, supersonic) differ only in attribute values, so they are modelled as a `category` attribute rather than as subtypes. Similarly, real and synthetic flights share all attributes and are distinguished by `data_source`.

4.5 Logical Design

The ER model maps to a property graph: each entity becomes a node label, each relationship a relationship type with its attributes as properties, and the M:N relationships need no junction tables. The property types below are read from the database itself (`db.schema.nodeTypeProperties()`); a star (*) marks a property present on every node of the label.

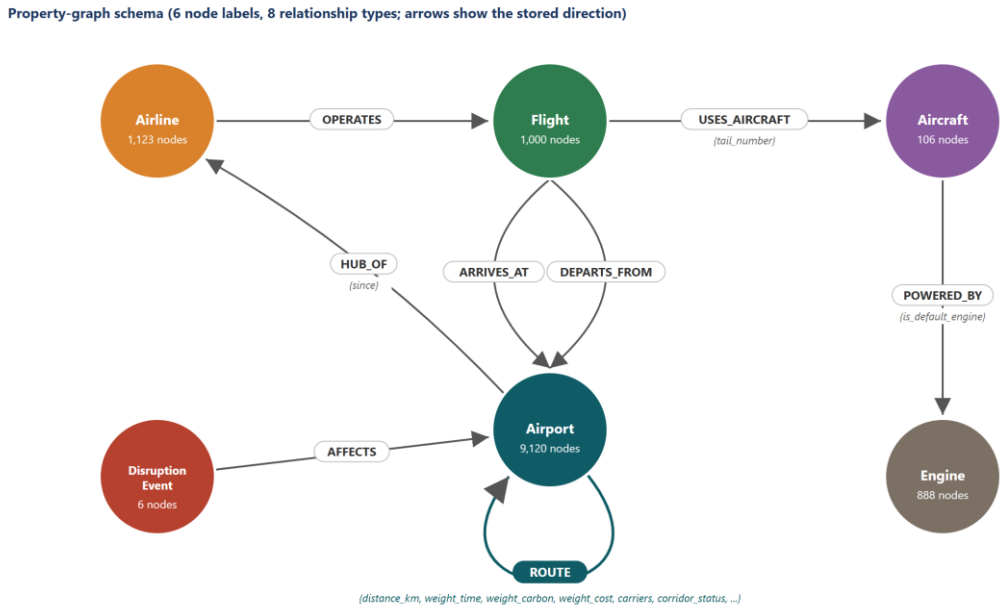


Figure 4: Property-graph schema (logical design)

Node label :Airport (9,120 nodes)

Table 15: Logical design of :Airport

Property	Data type	Size	Constraint	Description
`airport_id` *	STRING	3	UNIQUE	IATA code (key)
`city`	STRING	var.	-	City served
`country` *	STRING	2	-	ISO country code
`daily_capacity` *	INTEGER	var.	-	Daily movement capacity (500 for large airports, 100 otherwise)
`elevation_m`	FLOAT	var.	-	Elevation in metres
`is_hub` *	BOOLEAN	var.	-	Large airport
`location` *	POINT	point	IS :: POINT	WGS-84 point (latitude, longitude)
`name` *	STRING	var.	NOT NULL	Name
`operational_status` *	STRING	var.	-	ACTIVE / RESTRICTED / CLOSED (soft state)
`region` *	STRING	var.	-	Continent
`runway_count` *	INTEGER	var.	-	Number of runways
`timezone`	STRING	var.	-	IANA time zone

Node label :Airline (1,123 nodes)

Table 16: Logical design of :Airline

Property	Data type	Size	Constraint	Description
`airline_id` *	STRING	2	UNIQUE	IATA code (key)
`country`	STRING	2	-	ISO country code
`fleet_size` *	INTEGER	var.	-	Fleet size (synthetic)
`name` *	STRING	var.	-	Name

Node label :Aircraft (106 nodes)

Table 17: Logical design of :Aircraft

Property	Data type	Size	Constraint	Description
aircraft_type_id *	STRING	3-4	UNIQUE	ICAO type code (key)
avg_fuel_burn_kg_per_km *	FLOAT	var.	-	Cruise fuel burn (synthetic, size-scaled)
category *	STRING	var.	-	Aircraft category
engine_count *	INTEGER	var.	-	Number of engines
manufacturer *	STRING	var.	-	Manufacturer
max_range_km *	INTEGER	var.	-	Range
model *	STRING	var.	-	Model name
seating_capacity *	INTEGER	var.	-	Certified maximum seats
typical_seats *	INTEGER	var.	-	Seats in a typical layout

Node label :Engine (888 nodes)

Table 18: Logical design of :Engine

Property	Data type	Size	Constraint	Description
bypass_ratio	FLOAT	var.	-	Bypass ratio
engine_model *	STRING	var.	-	Engine identification
engine_uid *	STRING	~8	UNIQUE	Databank UID (key)
fuel_lto_cycle_kg	FLOAT	var.	-	Fuel per landing/take-off cycle
manufacturer *	STRING	var.	-	Manufacturer
rated_thrust_kn *	FLOAT	var.	-	Rated thrust (kN)

Node label :Flight (1,000 nodes)

Table 19: Logical design of :Flight

Property	Data type	Size	Constraint	Description
actual_arrival	ZONED DATETIME	var.	-	Actual arrival
actual_departure	ZONED DATETIME	var.	-	Actual departure
base_fare_usd *	FLOAT	var.	-	Indicative fare (synthetic)
data_source *	STRING	var.	-	BTS_2015 / SYNTHETIC / MANUAL
delay_minutes	INTEGER	var.	-	Departure delay
distance_km *	FLOAT	var.	-	Distance
duration_min *	INTEGER	var.	-	Scheduled duration
flight_id *	STRING	~15	UNIQUE	Carrier + number + date (key)
flight_number *	STRING	var.	-	Flight number
scheduled_arrival *	ZONED DATETIME	var.	-	Scheduled arrival (zoned)
scheduled_departure *	ZONED DATETIME	var.	IS :: ZONED DATETIME	Scheduled departure (zoned)
status *	STRING	var.	NOT NULL	SCHEDULED / COMPLETED / DELAYED / CANCELLED / DIVERTED

Node label :DisruptionEvent (6 nodes)

Table 20: Logical design of :DisruptionEvent

Property	Data type	Size	Constraint	Description
data_source *	STRING	var.	-	BTS_2015 / SYNTHETIC / MANUAL
description *	STRING	var.	-	Description
end_time *	ZONED DATETIME	var.	-	End (null while active)
event_id *	STRING	~25	UNIQUE	Event id (key)
event_type *	STRING	var.	-	WEATHER / ATC_FAILURE / FOG / ...
impact_affected_trips	INTEGER	var.	-	Impact: sampled trips affected
impact_avg_extra_min	INTEGER	var.	-	Impact: average extra minutes
impact_cut_off_airports	INTEGER	var.	-	Impact: airports cut off
impact_flights_per_day	INTEGER	var.	-	Impact: daily flights lost
impact_lost_trips	INTEGER	var.	-	Impact: trips with no route
impact_seats_per_day	INTEGER	var.	-	Impact: daily seats lost
severity *	STRING	var.	-	MODERATE / SEVERE / CRITICAL
start_time *	ZONED DATETIME	var.	-	Start

Relationship type :ROUTE (56,627 relationships)

Table 21: Logical design of :ROUTE

Property	Data type	Size	Constraint	Description
aircraft_types *	LIST<STRING> NOT NULL	var.	-	Aircraft flown on the route
avg_duration_min *	INTEGER	var.	-	Average flight time
avg_speed_kmh *	FLOAT	var.	-	Average block speed
carriers *	LIST<STRING> NOT NULL	var.	-	Airlines on the route
co2_emissions_kg *	FLOAT	var.	-	CO2 per flight
co2_per_seat_kg *	FLOAT	var.	-	CO2 per seat
congestion_index *	FLOAT	var.	-	Congestion indicator (synthetic)
corridor_status *	STRING	var.	-	ACTIVE / DISRUPTED / CLOSED
distance_km *	FLOAT	var.	-	Distance
duration_estimated *	BOOLEAN	var.	-	Duration replaced by the model
fuel_burn_estimate_kg *	FLOAT	var.	-	Fuel per flight incl. LTO
fuel_burn_source *	STRING	var.	-	How the aircraft were determined
last_updated *	ZONED DATETIME	var.	-	Last change
weight_carbon *	FLOAT	var.	IS :: FLOAT	Weight = co2_per_seat_kg
weight_cost *	FLOAT	var.	-	Weight = 50 + 0.10 x km
weight_time *	FLOAT	var.	IS :: FLOAT	Weight = avg_duration_min

Relationship type :POWERED_BY (110 relationships)

Table 22: Logical design of :POWERED_BY

Property	Data type	Size	Constraint	Description
is_default_engine *	BOOLEAN	var.	-	Default engine option

Relationship type :USES_AIRCRAFT (1,000 relationships)

Table 23: Logical design of :USES_AIRCRAFT

Property	Data type	Size	Constraint	Description
`tail_number`	STRING	var.	-	Aircraft registration

`OPERATES`, `DEPARTS_FROM`, `ARRIVES_AT`, `AFFECTS` and `HUB_OF` carry no properties (`HUB_OF.since` is part of the design, but no source provides it).

4.6 Normalization

Normal forms are defined for relations, but the same reasoning applies to node properties: every property of a node should depend on the node's key, and nothing should depend on a non-key property. Repeating groups (many routes per airport, many engines per aircraft) are relationships rather than repeated properties, so the node labels are in third normal form.

4.6.1 Functional Dependencies

Table 24: Functional dependencies

Label / relationship	Functional dependencies
Airport	airport_id -> name, city, country, region, location, timezone, elevation_m, runway_count, is_hub, daily_capacity, operational_status
Airline	airline_id -> name, country, fleet_size
Aircraft	aircraft_type_id -> model, manufacturer, category, seating_capacity, max_range_km, engine_count, typical_seats, avg_fuel_burn_kg_per_km; (category, seating_capacity) -> typical_seats
Engine	engine_uid -> engine_model, manufacturer, bypass_ratio, rated_thrust_kn, fuel_lto_cycle_kg
Flight	flight_id -> flight_number, scheduled and actual times, status, delay_minutes, distance_km, duration_min, data_source; (scheduled_departure, actual_departure) -> delay_minutes -> status
ROUTE	(source airport, target airport) -> distance_km, avg_duration_min, aircraft_types, carriers, fuel_burn_estimate_kg, co2_per_seat_kg, weights, corridor_status

4.6.3 Justified Denormalization

Some dependencies are deliberately stored redundantly because the queries need them at traversal speed:

- **Precomputed route weights.** `co2_per_seat_kg`, `fuel_burn_estimate_kg` and the three `weight_*` properties are derived from distance, the aircraft mix and engine data. Storing them on the edge lets Dijkstra and the itinerary search read one property per hop instead of joining four labels.
- **Lists on ROUTE.** `carriers` and `aircraft_types` are multi-valued attributes stored as lists instead of separate relationships (which would add about 200,000 relationships) because they are read together and never queried individually during pathfinding.
- **Copies on Flight.** `distance_km` and `duration_min` duplicate route values so a flight remains meaningful if a route is later changed.
- **Derived delay and status.** `delay_minutes` and `status` are derived from times but stored, so aggregations (Query 7) do not recompute them.

- **Impact summary on events.** Six `impact_*` properties store the headline numbers of a simulation so the event history can show them without recomputation.

These derived values are always written by the same code that computes them (`seed3.py`, `crud.record_departure`, `queries.simulate_disruption`), which keeps them consistent.

4.7 Physical Design

4.7.1 Data Types and Storage

Neo4j stores nodes and relationships in fixed-size records that point directly to their neighbours (index-free adjacency), with properties in a separate property store; frequently used pages are cached in the 1 GB page cache. The project uses Neo4j's native types: STRING for codes and text, INTEGER and FLOAT for measures, BOOLEAN flags, LIST<STRING> for carriers and aircraft, POINT (WGS-84) for locations, and ZONED DATETIME for all times, so time-zone arithmetic happens in the database. The GDS projection keeps a compressed in-memory copy of the active network (3,670 nodes, 56,627 relationships, five relationship weights and two node coordinates) for the algorithms.

4.7.2 Indexing Strategy

Table 25: Indexes (18 in total)

Index	Type	Supports
airport_id, airline_id, aircraft_type_id, engine_uid, flight_id, event_id	Range (backing the unique constraints)	Key lookups in every query and CRUD call
airport_region, airport_status, airport_country	Range	Region closures, status filters, listing by country
airport_location	Point	Spatial radius search (Query 9)
airport_search (name, city, airport_id)	Full-text (Lucene)	Airport autocomplete in the web application
flight_status, flight_departure, flight_data_source	Range	Flight filters and event time windows (Query 8)
route_status (ROUTE.corridor_status)	Relationship range	Filtering active corridors
event_start (DisruptionEvent.start_time)	Range	Event history ordered by time
2 token lookup indexes	Lookup	Label and relationship-type scans (Neo4j default)

Indexes were chosen for properties used to find starting points of traversals; once a start node is found, traversal follows relationships without index lookups. The measured effect is in Section 6.2.

4.7.3 Partitioning

Not used. The database (about 220 MB) fits on one machine, and graph traversals are fastest when the whole graph is local. Neo4j Enterprise could split data across databases with composite databases if the flight layer grew to a full year of global flights.

4.8 Integrity Constraints

Table 26: Integrity constraints

Relational concept	Graph implementation in this project
Primary key	Unique constraints on the six keys (`airport_id`, `airline_id`, `aircraft_type_id`, `engine_uid`, `flight_id`, `event_id`).
Foreign key	Relationships: a relationship can only connect two existing nodes, so a flight cannot reference a missing airport. `crud.create_flight` creates the flight and its four relationships in one transaction.
Unique	The same six constraints; a duplicate insert fails with `ConstraintValidationFailed` (checked by the API test, Section 7.2).
Not null	Existence constraints: `Airport.name IS NOT NULL`, `Flight.status IS NOT NULL`.
Check	Property type constraints (`Airport.location IS :: POINT`, `Flight.scheduled_departure IS :: ZONED DATETIME`, `ROUTE.weight_time` and `weight_carbon IS :: FLOAT`) plus API validation: code patterns such as `^[A-Z0-9]{3}$`, value ranges for latitude and longitude, and allowed status values.
Default	Applied in Cypher: new airports and corridors start `ACTIVE`, new flights `SCHEDULED`; re-seeding keeps existing statuses with `coalesce`.
Referential actions	`DETACH DELETE` deletes a flight or event with its relationships (cascade). Deleting an airport that still has routes is refused (restrict); airports are closed instead.

Chapter 5: Implementation

5.1 Development Environment and Setup

Development used Windows 11 laptops with Neo4j Desktop 2, Visual Studio Code and a Python virtual environment. The DBMS "DBMS Airlines" (Neo4j 2026.09 Enterprise) runs locally with the GDS plugin enabled; its heap was raised to 4 GB and the page cache to 1 GB after an out-of-memory error during testing (Section 5.9).

Setup steps:

1. Install Neo4j Desktop, create a DBMS (2026.09 Enterprise), install the Graph Data Science plugin and start the DBMS.
2. Create the virtual environment and install dependencies: `python -m venv .venv` and `.venv/Scripts/python -m pip install -r requirements.txt`.
3. Create `.env` with `NEO4J_PASSWORD`, `ADMIN_USER`, `ADMIN_PASSWORD` and `SESSION_SECRET`.
4. Download `flights.csv` from Kaggle into the project folder (the other datasets are in the repository).
5. Load the data in order: `seed.py`, `seed2.py`, `seed3.py`, `seed4.py`, `seed5.py` (each also has a `--dry-run` mode that validates without a database).
6. Start the web application: `.venv/Scripts/python -m uvicorn app:app` and open http://localhost:8000.

Data loading pipeline. The five scripts read each dataset with pandas, clean it, and load it with batched (500 rows), parameterised `MERGE` statements, after creating the constraints. Key cleaning steps: filtering malformed and duplicate IATA codes; excluding 33 airport codes whose route-data coordinates were more than 50 km from the OurAirports location; replacing impossible durations with a block-time model (46.2 + 0.0725 x km minutes); converting local flight times to zoned date-times; choosing each route's aircraft from the most direct evidence available (carrier equipment on the route 35.7%, any carrier on the pair 10.1%, the carrier's fleet on similar distances 42.6%, a distance band 11.6%); and detecting real disruption events from spikes in weather and air-traffic cancellations.

5.2 DBMS, Tools and Libraries Used

The back end is built with FastAPI [21]; the browser uses Leaflet [22] for maps and vis-network [23] for the graph view.

Table 27: Tools and libraries

Category	Tool	Use in the project
DBMS	Neo4j 2026.09 Enterprise (Neo4j Desktop 2)	Storage, Cypher, transactions, constraints, indexes
Analytics	Graph Data Science 2026.09.0	Dijkstra (single-pair and single-source), A*, Yen's, Betweenness, PageRank
Console	Neo4j Browser	Ad-hoc Cypher, schema visualisation, graph views
Language	Python 3.10	Pipeline and back end
Data	pandas, openpyxl, tzdata / zoneinfo	Cleaning, Excel input, time zones
Connectivity	neo4j Python driver 6.x (Bolt)	Managed and explicit transactions
Web back end	FastAPI, Pydantic, Uvicorn, Starlette middleware, itsdangerous	REST API, validation, sessions, gzip
Front end	HTML5, CSS3, JavaScript; Leaflet 1.9.4; vis-network 9.1.9; HTML canvas; SVG	Pages, maps, graph, live traffic, charts
Testing	FastAPI TestClient with httpx; headless browser	57-check API test, screenshots and layout checks
Version control	Git, GitHub, GitHub CLI	Source code and history

5.5 Views and Indexes Created

Neo4j has no SQL views. Their role is filled by two mechanisms:

- **The GDS projection** `active_network` is a virtual, in-memory view of the operational network: only ACTIVE airports joined by ACTIVE corridors, with five weights. It is rebuilt after every change of status, so algorithms always see the current network. Its definition is a Cypher aggregation projection:

```
MATCH (s:Airport {operational_status: 'ACTIVE'})-[r:ROUTE {corridor_status: 'ACTIVE'}]->
      (t:Airport {operational_status: 'ACTIVE'})
WITH gds.graph.project($graph, s, t, {
       sourceNodeProperties: {latitude: s.location.latitude, longitude: s.location.longitude},
       targetNodeProperties: {latitude: t.location.latitude, longitude: t.location.longitude},
       relationshipProperties: r {.weight_time, .weight_carbon, .weight_cost, .distance_km,
                                  weight_trip: r.weight_time + $connection_min}
     }) AS g
RETURN g.graphName AS graph, g.nodeCount AS nodes, g.relationshipCount AS relationships
```

- **Cypher map projections** such as `a {.*}` and `r {.weight_time, .weight_carbon}` return a chosen view of a node or relationship to the application.

The 12 constraints and 18 indexes are listed in Sections 4.7.2 and 4.8; their DDL statements are in Appendix A.

5.7 Application Connectivity

The application connects to Neo4j through the official neo4j Python driver over the Bolt binary protocol (no JDBC, ODBC or ORM). One driver instance is created when FastAPI starts, with short timeouts so a stopped database produces a clear error, and every request borrows a session from its connection pool. Single statements run as managed transactions, which the driver retries on transient errors:

```
def _write(driver, query, **params):
    with driver.session() as session:
        return plain(session.execute_write(lambda tx: tx.run(query, **params).data()))


def create_airline(driver, airline_id, name, country, fleet_size=None, alliance=None):
    return _one(_write(driver, """
        CREATE (al:Airline {airline_id: $airline_id, name: $name, country: $country,
```

```
                        fleet_size: $fleet_size, alliance: $alliance})
        RETURN al {.*} AS airline""", airline_id=airline_id, name=name, ...))
```

Multi-step operations use explicit transactions (`session.begin_transaction()` … `tx.commit()`), for example the disruption closure in Section 6.1. FastAPI exposes the functions as 46 REST endpoints returning JSON; Neo4j errors are translated to HTTP status codes (409 for constraint violations, 503 if Neo4j is unreachable). The browser calls the endpoints with `fetch()`.

Table 28: REST API summary

Area	Endpoints (method and path)
Public	GET `/api/stats`, `/api/airports/search`, `/api/routes/find`, `/api/graph/airport|flight|event/{id}`, `/api/analytics[/{n}]`, `/api/events`, `/api/flights`, `/api/live`, `/api/replay`
Authentication	POST `/api/login`, `/api/logout`; GET `/api/me`
Admin CRUD	POST/PUT/DELETE on `/api/airports`, `/api/airlines`, `/api/flights` (+ departure, aircraft), `/api/routes` (+ status), `/api/hubs`, `/api/events` (+ resolve)
Disruptions	POST `/api/disruptions`, POST `/api/disruptions/{id}/restore`

5.8 Version Control and Project Repository

Repository: https://github.com/Karthick-Ganesh/dbms-airline-graph

The repository contains the data pipeline, application, web pages, datasets (except the 590 MB flights file, which is downloaded from Kaggle), the README and the design document (6,751 lines of code in total). Credentials (`.env`) and local settings are excluded through `.gitignore`.

Table 29: Main commits

Commit	Date	Change
582661d	2026-09-24	Initial commit with project rubrics
28611d4 / aa1cb71	2026-09-24	ICAO engine emissions data and the first loader
cbbcd15	2026-09-25	Full airline graph, CRUD, analytical queries, Review 2 demo
11a25b6	2026-10-25	Neo Airways web application
2e6973f	2026-10-25	Live traffic page, map wrap-around fix
aef444c	2026-10-25	Bookable route search, realistic CO2 per seat, lighter GDS refreshes

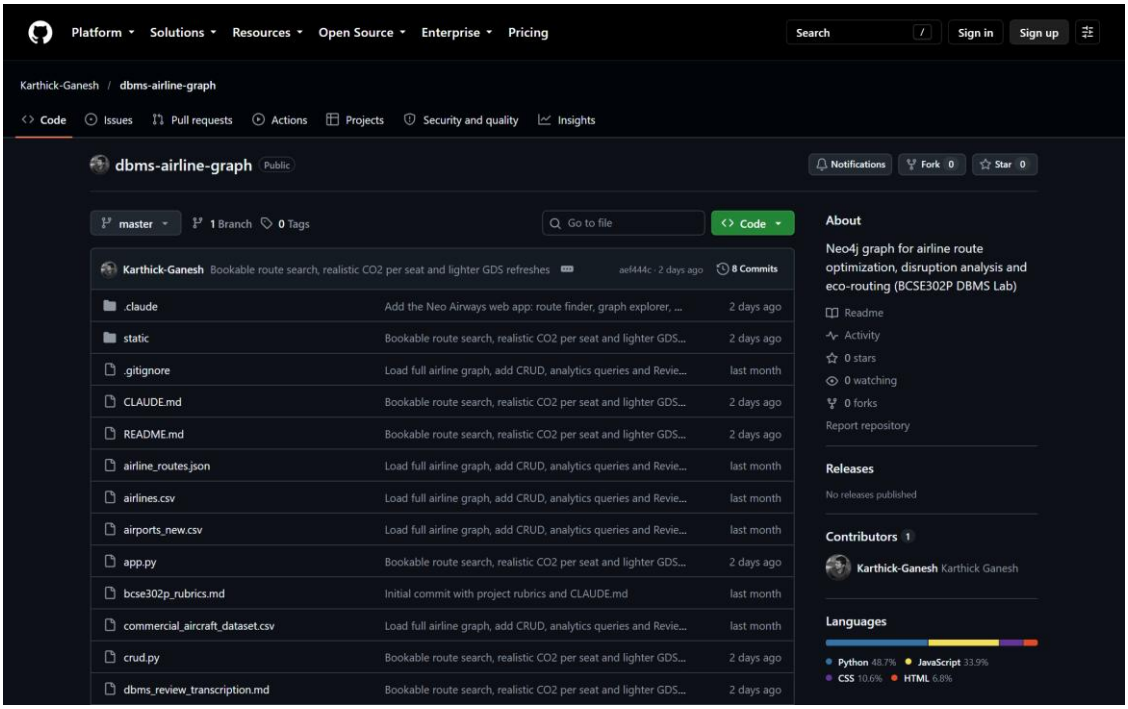


Figure 5: Project repository on GitHub

5.9 Challenges Faced and Solutions Adopted

Table 30: Challenges and solutions

Challenge	Solution
Duplicate and reused IATA codes across datasets (e.g. old airports reusing MUC, HKG)	Filter code formats, rank candidate rows (in-service, scheduled service) and keep one deliberately; never let MERGE keep the last row read
Pandas reading Namibia's code "NA" as a missing value	Read every CSV with `keep_default_na=False`
33 airport codes meaning different airports in two datasets (e.g. IAL)	Compare coordinates; exclude codes more than 50 km apart
Route data without aircraft for half of the routes	Four-tier inference recorded in `fuel_burn_source`
Impossible durations and self-loops in route data	Block-time model and filtering
Local times without dates in the flight data	Compute arrivals in UTC and convert with each airport's time zone; matches the dataset for 99.99% of rows
Greenest path chaining 7 short hops	Charge every leg one landing/take-off cycle from ICAO engine data
"Fastest" one-stop beating a nonstop	Add 90 minutes per connection (`weight_trip`)
Large aircraft looking unrealistically green (maximum seats)	Introduce `typical_seats` and size-scaled fuel burn; recompute all route weights
GDS Dijkstra cannot limit the number of stops	Exact Cypher enumeration of 0-, 1- and 2-stop itineraries with a UNION of fixed-length patterns
Variable-length search taking 2-5 s	Fixed-length patterns let the planner join from both ends: 0.1-0.9 s
Neo4j out of memory after many rapid re-projections	Raise heap to 4 GB; drop the projection after changes and rebuild once on next use
10,000 moving aircraft in the browser	Draw on one canvas instead of map markers; no frame over 50 ms
Routes crossing the date line detached from their airports	Draw lines and markers in three world copies

Chapter 6: Query Design and Performance Tuning

Cypher queries are written as patterns. The table below maps the SQL concepts named in the template to their Cypher equivalents used in this project, and the subsections show representative queries with their results.

Table 31: SQL concepts and their Cypher equivalents

SQL concept	Cypher equivalent	Example in this project
Selection (WHERE)	MATCH with property maps and WHERE	Query 9 (spatial), CRUD reads
Join	Pattern matching across relationships	Queries 8 and 11 (four labels)
Grouping and aggregation	Implicit grouping keys with count, avg, sum, collect	Queries 2, 7, 10, 12
Subqueries	CALL { } subqueries, COUNT { } and EXISTS { }	Queries 6 and 12, itinerary legs
Set operations	UNION / UNION ALL	Bookable itinerary search
Recursive queries	Variable-length paths, shortestPath, GDS procedures	Queries 2, 3, 4, 13, 14

6.1 Representative Queries

6.1.1 Selection with a spatial predicate

Purpose: Find alternative airports within 300 km of Chennai, using the point index.

```
MATCH (c:Airport {airport_id: $center})
WITH c.location AS here
MATCH (a:Airport)
WHERE point.distance(a.location, here) < $metres AND a.location <> here
RETURN a.airport_id AS airport, a.name AS name, a.city AS city,
       round(point.distance(a.location, here) / 1000) AS km,
       COUNT { (a)-[:ROUTE]->() } AS routes
ORDER BY km LIMIT 10
```

Result (parameters `center = MAA`, `metres = 300000`; 5 ms):

airport	name	city	km	routes
TIR	Tirupati Airport	Tirupati	99	7
PNY	Pondicherry Airport	Pondicherry	120	2
NVY	Neyveli Airport	Neyveli	168	0
CDP	Kadapa Airport	Kadapa	227	3
SXV	Salem Airport	Salem	265	4
BLR	Kempegowda International Airport	Bangalore	268	98
TJV	Tanjore Air Force Base	Thanjavur	278	0
PUT	Sri Sathya Sai Airport	Puttaparthi	288	0

Insight: If Chennai closes, Bengaluru (268 km, 98 routes) is the only nearby airport with the capacity to take diverted traffic.

6.1.2 Join across four labels

Purpose: Compare the engines of the aircraft flying Delhi-Mumbai: ROUTE -> aircraft list -> Aircraft -POWERED_BY-> Engine.

```
MATCH (:Airport {airport_id: $src})-[r:ROUTE]->(:Airport {airport_id: $dst})
UNWIND r.aircraft_types AS type_id
MATCH (ac:Aircraft {aircraft_type_id: type_id})-[p:POWERED_BY]->(e:Engine)
```

```
RETURN ac.model AS aircraft, e.engine_model AS engine, e.manufacturer AS maker,
       round(e.bypass_ratio, 2) AS bypass_ratio, round(e.fuel_lto_cycle_kg) AS lto_fuel_kg,
       p.is_default_engine AS default_engine
ORDER BY bypass_ratio DESC
```

Result (parameters `src = DEL`, `dst = BOM`; 11 ms):

aircraft	engine	maker	bypass_ratio	lto_fuel_kg	default_engine
787-8	Trent 1000-AE3	Rolls-Royce plc	9.14	884	false
787-8	GEnx-1B64	General Electric Company	9	796	true
777-300ER	GE90-115B	General Electric Company	6.96	1,449	true
A319	CFM56-5B5/3	CFM International	6	343	true
A320	CFM56-5B4	CFM International	5.90	421	true

Insight: Higher bypass ratios burn less fuel. An aircraft type can be certified with several engines, which is why POWERED_BY is many-to-many.

6.1.3 Grouping and aggregation

Purpose: On-time performance per airline, separating real and synthetic flights.

```
MATCH (al:Airline)-[:OPERATES]->(f:Flight)
WITH al, f.data_source AS source, count(f) AS flights,
     avg(f.delay_minutes) AS avg_delay,
     sum(CASE f.status WHEN 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
     sum(CASE f.status WHEN 'DELAYED' THEN 1 ELSE 0 END) AS delayed
WHERE flights >= $min_flights
RETURN al.airline_id AS airline, al.name AS name, source, flights,
       round(avg_delay, 1) AS avg_delay_min,
       round(100.0 * delayed / flights, 1) AS delayed_pct,
       round(100.0 * cancelled / flights, 1) AS cancelled_pct
ORDER BY cancelled_pct DESC, delayed_pct DESC LIMIT 12
```

Result (parameters `min_flights = 10`; 6 ms):

airline	name	source	flights	avg_delay_min	delayed_pct	cancelled_pct
B6	JetBlue Airways	BTS_2015	65	17.10	20	36.90
UA	United Airlines	BTS_2015	54	12.90	20.40	29.60
MQ	American Eagle Airlines	BTS_2015	39	29	30.80	28.20
US	US Airways	BTS_2015	64	4.40	12.50	28.10
DL	Delta Air Lines	BTS_2015	100	10.10	13	21

Insight: JetBlue's high cancellation rate reflects its New York base during Winter Storm Juno (the sample includes 26-28 January).

6.1.4 Aggregation with a constant applied in Cypher

Purpose: Carbon efficiency of each aircraft category (3.16 kg CO2 per kg of fuel, typical seats).

```
MATCH (ac:Aircraft) WHERE ac.typical_seats > 0
WITH ac.category AS category, count(*) AS types,
     avg(ac.avg_fuel_burn_kg_per_km * 3.16 / ac.typical_seats * 1000) AS g_co2_per_seat_km
RETURN category, types, round(g_co2_per_seat_km, 1) AS g_co2_per_seat_km
ORDER BY g_co2_per_seat_km
```

Result (0 ms):

category	types	g_co2_per_seat_km
Narrowbody	33	64.80
Turboprop	16	72.60
Widebody	27	74.70
Regional	25	96.80
Supersonic	1	499.60

Insight: With realistic seat counts and size-scaled fuel burn, narrowbodies are the most efficient per seat (about 65 g CO2 per seat-km); private jets emit about eight times more.

6.1.5 Subquery (COUNT { }) with aggregation

Purpose: How much of each airline's network depends on its hub airports: a resilience indicator.

```
MATCH ()-[r:ROUTE]->()
UNWIND r.carriers AS carrier
WITH carrier, count(*) AS all_routes
WHERE all_routes >= $min_routes
MATCH (hub:Airport)-[:HUB_OF]->(al:Airline {airline_id: carrier})
WITH al, all_routes, hub,
     COUNT { (hub)-[h:ROUTE]->() WHERE al.airline_id IN h.carriers } AS hub_routes
WITH al, all_routes, collect(hub.airport_id) AS hubs, sum(hub_routes) AS via_hubs
RETURN al.airline_id AS airline, al.name AS name, all_routes, hubs,
       round(100.0 * via_hubs / all_routes, 1) AS pct_routes_from_hubs
ORDER BY pct_routes_from_hubs DESC LIMIT 10
```

Result (parameters `min_routes = 100`; 73 ms):

airline	name	all_routes	hubs	pct_routes_from_hubs
VA	Virgin Australia	163	ADL, BNE, MEL, PER, SYD	54
W9	Wizz Air UK	123	LGW, LTN	53.70
F3	Flyadeal	118	DMM, JED, MED, RUH	53.40
J9	Jazeera Airways	179	DMM, KWI	52
3O	Air Arabia Maroc	108	CMN, FEZ, NDR, RBA, TNG	50.90

Insight: Single-hub carriers such as Etihad (AUH) and El Al (TLV) lose half of their network if one airport closes.

6.1.6 Set operation: the bookable itinerary search (UNION ALL)

The route finder enumerates every nonstop, 1-stop and 2-stop itinerary over active corridors with three fixed-length patterns joined by `UNION ALL` inside a `CALL` subquery. Intermediate airports must lie within twice the direct distance. The application then picks the fastest itinerary, and the greenest and shortest within the extra-time limit.

```
MATCH (s:Airport {airport_id: $src}), (t:Airport {airport_id: $dst})
WITH s, t, $detour * point.distance(s.location, t.location) AS budget
CALL (s, t, budget) {
  MATCH (s)-[r1:ROUTE {corridor_status: 'ACTIVE'}]->(t) RETURN [s.airport_id, t.airport_id] AS
path, [r1] AS rs
  UNION ALL
  MATCH (s)-[r1:ROUTE {corridor_status: 'ACTIVE'}]->(m1:Airport)-[r2:ROUTE {corridor_status:
'ACTIVE'}]->(t)
    WHERE m1.operational_status = 'ACTIVE' AND point.distance(m1.location, s.location) +
point.distance(m1.location, t.location) <= budget
    RETURN [s.airport_id, m1.airport_id, t.airport_id] AS path, [r1, r2] AS rs
  UNION ALL
  MATCH (s)-[r1:ROUTE {corridor_status: 'ACTIVE'}]->(m1:Airport)-[r2:ROUTE {corridor_status:
'ACTIVE'}]->(m2:Airport)-[r3:ROUTE {corridor_status: 'ACTIVE'}]->(t)
```

```
    WHERE m1 <> t AND m2 <> s AND m1.operational_status = 'ACTIVE' AND point.distance(m1.location,
s.location) + point.distance(m1.location, t.location) <= budget AND m2.operational_status =
'ACTIVE' AND point.distance(m2.location, s.location) + point.distance(m2.location, t.location) <=
budget
    RETURN [s.airport_id, m1.airport_id, m2.airport_id, t.airport_id] AS path, [r1, r2, r3] AS rs
}
RETURN path, size(rs) AS legs,
       reduce(m = 0, r IN rs | m + r.avg_duration_min) + $connection * (size(rs) - 1) AS journey,
       reduce(c = 0.0, r IN rs | c + r.co2_per_seat_kg) AS co2,
       reduce(k = 0.0, r IN rs | k + r.distance_km) AS km
```

Table 32: Leh (IXL) to Lisbon (LIS): 37 itineraries checked in 140 ms

Option	Itinerary	Journey	CO2 / seat	vs typical
fastest	IXL -> DEL -> CDG -> LIS	16 h 10 m	658.3 kg	-5%
greenest	IXL -> IXC -> DXB -> LIS	17 h 13 m	600.8 kg	-13%
shortest	IXL -> DEL -> FCO -> LIS	16 h 15 m	728.8 kg	+5%

The unconstrained GDS Dijkstra optimum for the same trip is IXL -> DEL -> TBS -> ATH -> LIS (3 stops, 19 h 50 m, 557.9 kg): greener, but with more stops than a passenger would book.

6.1.7 Graph algorithms (GDS)

Betweenness centrality: critical transfer airports

Purpose: Identify single points of failure: airports on the most shortest paths between other airports.

```
CALL gds.betweenness.stream($graph, {samplingSize: 1000, samplingSeed: 42})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS a, score
RETURN a.airport_id AS airport, a.city AS city, a.country AS country, round(score) AS betweenness
ORDER BY betweenness DESC LIMIT 10
```

Result (293 ms):

airport	city	country	betweenness
IST	Arnavutköy, Istanbul	TR	219,588
ANC	Anchorage	US	210,842
DXB	Dubai	AE	201,495
CDG	Paris	FR	168,162
FRA	Frankfurt am Main	DE	165,798

Insight: Anchorage ranks second despite modest traffic: it is the bridge between Alaska's many small airports and the rest of the network, the "anomalous centrality" described by Guimerà et al. [10].

Yen's k-shortest paths: alternatives

Purpose: The three fastest Delhi-New York itineraries, as a recommendation list.

```
MATCH (s:Airport {airport_id: $src}), (t:Airport {airport_id: $dst})
CALL gds.shortestPath.yens.stream($graph, {
  sourceNode: s, targetNode: t, k: $k, relationshipWeightProperty: 'weight_trip'
}) YIELD index, nodeIds, totalCost
RETURN index + 1 AS rank, [n IN gds.util.asNodes(nodeIds) | n.airport_id] AS path,
       size(nodeIds) - 1 AS flights, round(totalCost - $connection_min) AS journey_minutes
ORDER BY rank
```

Result (parameters `src = DEL`, `dst = JFK`, `k = 3`, `connection_min = 90`; 30 ms):

rank	path	flights	journey_minutes
1	DEL -> JFK	1	1,010
2	DEL -> TAS -> JFK	2	1,060
3	DEL -> MUC -> JFK	2	1,123

Insight: The nonstop is fastest; one-stop alternatives via Tashkent and Munich follow within two hours.

6.1.8 Explicit transaction: closing airports

A closure sets the airport status, tags and disrupts every active corridor, and logs the event with its AFFECTS relationships, all in one explicit transaction. If any statement fails (for example a duplicate event id), Neo4j rolls back the whole transaction, as the Review 2 demonstration showed for LHR.

```
def _close_airports(tx, codes, event_id):
    """Soft-close ACTIVE airports and their ACTIVE corridors, tagging both with
    the event so restore_disruption() undoes exactly this closure."""
    closed = tx.run("""
        MATCH (a:Airport) WHERE a.airport_id IN $codes AND a.operational_status = 'ACTIVE'
        SET a.operational_status = 'CLOSED', a.disrupted_by = $event_id
        RETURN collect(a.airport_id) AS codes
        """, codes=codes, event_id=event_id).single()["codes"]
    tx.run("""
        MATCH (a:Airport)-[r:ROUTE]-() WHERE a.airport_id IN $codes AND r.corridor_status =
'ACTIVE'
        SET r.corridor_status = 'DISRUPTED', r.disrupted_by = $event_id
        """, codes=closed, event_id=event_id)
    return closed
```

6.2 Performance Tuning

Query plans were inspected with `PROFILE`, which reports the operators used and the number of database hits (record accesses). The comparison below shows why the indexes matter:

Table 33: PROFILE results

Case	Plan (operators)	DB hits	Time
Unique-index lookup on the key	Projection -> NodeUniqueIndexSeek	3	24.5 ms
Range index on region	EagerAggregation -> NodeIndexSeek	1,305	21.8 ms
No index (city): full label scan	Projection -> Filter -> NodeByLabelScan	17,505	27.2 ms
Full-text index	Projection -> ProcedureCall	2	21.2 ms
Point index (within 300 km of MAA)	EagerAggregation -> Filter -> Apply -> Projection -> NodeUniqueIndexSeek -> NodeIndexSeekByRange	14	33.0 ms

A lookup on an indexed key touches 3 records; the same kind of lookup on an unindexed property (city) scans all 9,120 airports (17,505 hits). Optimisations applied during development:

- **Fixed-length patterns instead of a quantified path.** The itinerary search with `(s)(()-[r:ROUTE]->()){1,3}(t)` took 2-5 s; three fixed-length patterns with `UNION ALL` let the planner expand from both ends and take 0.06-1.5 s, or 0.14-0.41 s for the measured trips (Table above).
- **Single-source Dijkstra for the impact report.** One `gds.allShortestPaths.dijkstra` call per origin solves all 50 destinations at once, instead of 1,250 separate searches.
- **Lazy projection rebuilds.** Writes drop the GDS projection; it is rebuilt once on the next query instead of after every write, which also solved an out-of-memory error.
- **Memory configuration.** Heap raised from 1 GB to 4 GB and page cache from 512 MB to 1 GB.

- **Caching and compression.** The live-traffic timetable (85,646 daily departures) is cached and rebuilt in the background; responses are gzip-compressed (656 KB to 182 KB).
- **Batched loading.** The seed scripts load 500 rows per `UNWIND` statement; re-seeding all 56,627 routes takes about 38 s.

Chapter 7: Results and Discussion

7.1 Output Screenshots and Sample Reports

The web application ("Neo Airways") presents each feature as a boarding pass. The screenshots were taken from the running system against the live database.

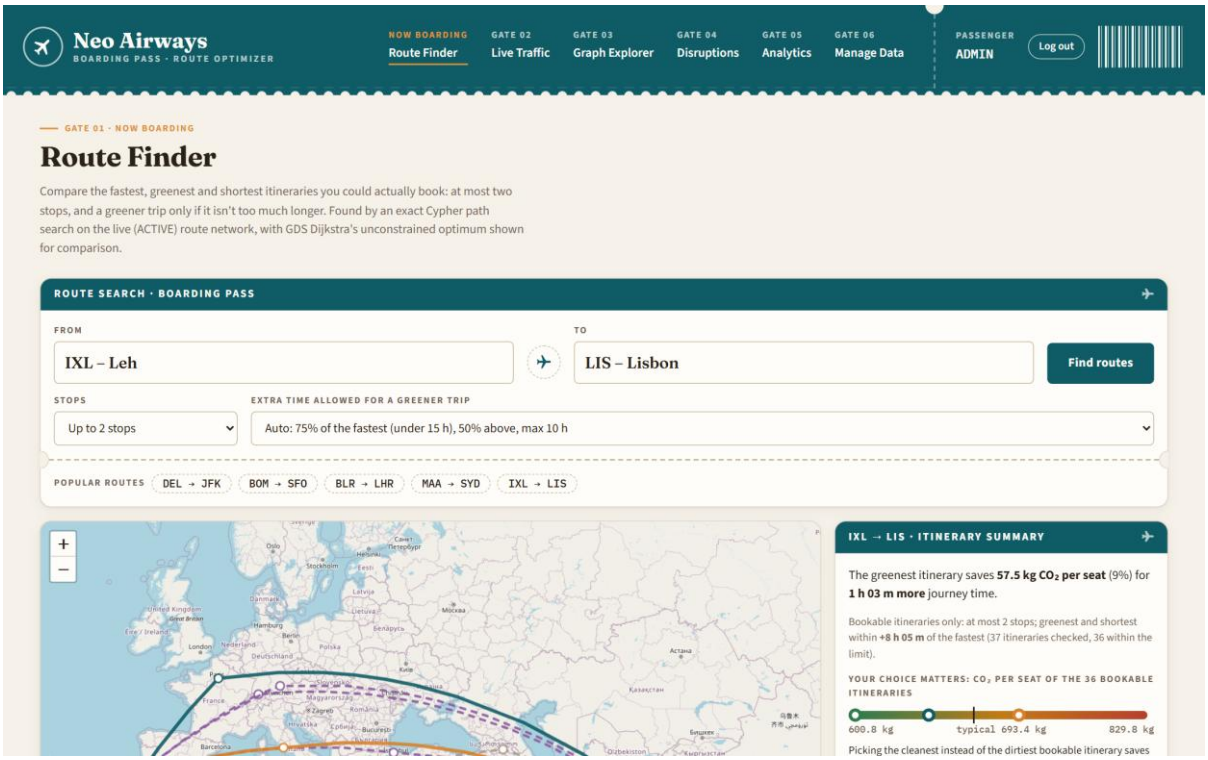


Figure 6: Route Finder: search with stop and extra-time limits

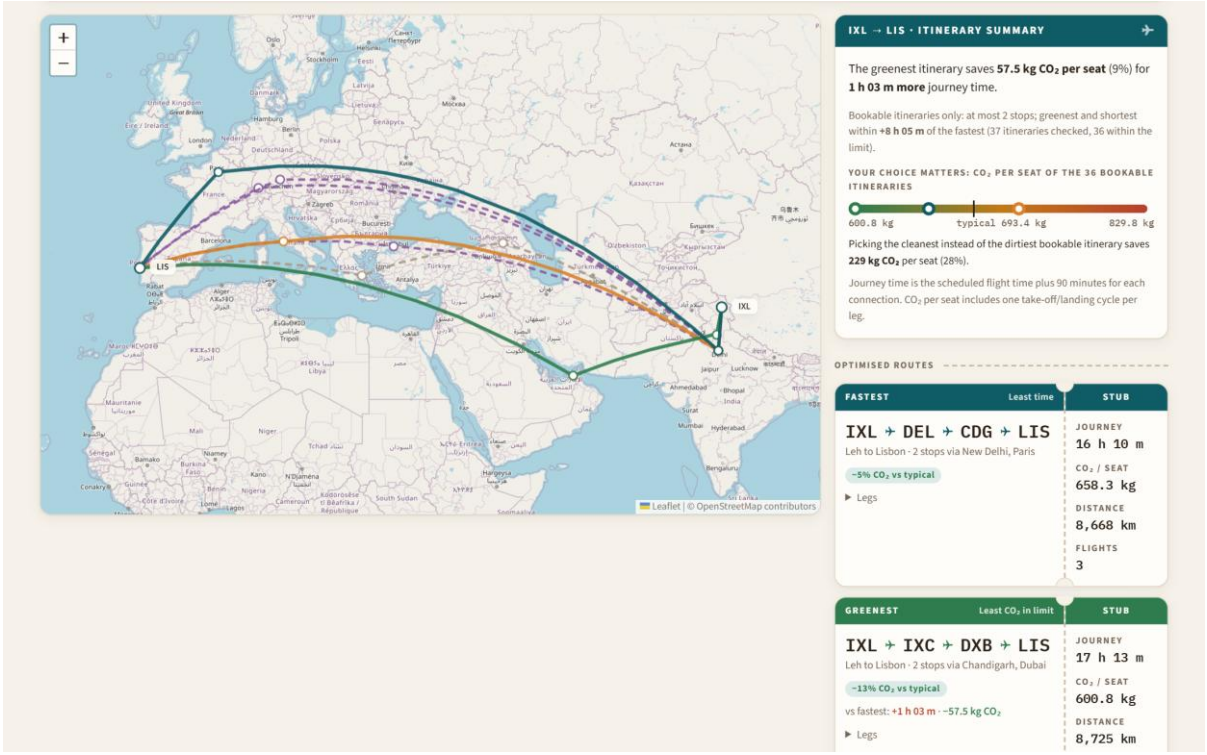


Figure 7: Route Finder: itineraries on the map and the CO2 range of all bookable choices

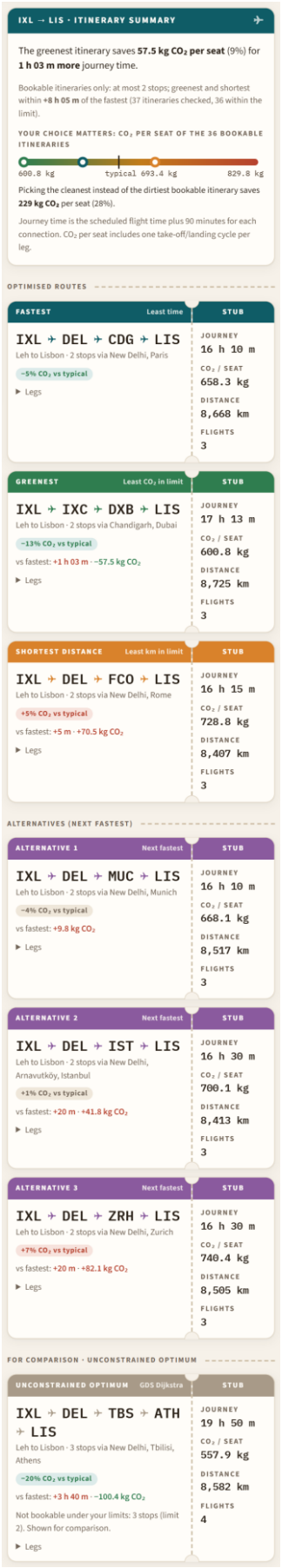


Figure 8: Route Finder: each itinerary as a boarding pass with a tear-off stub

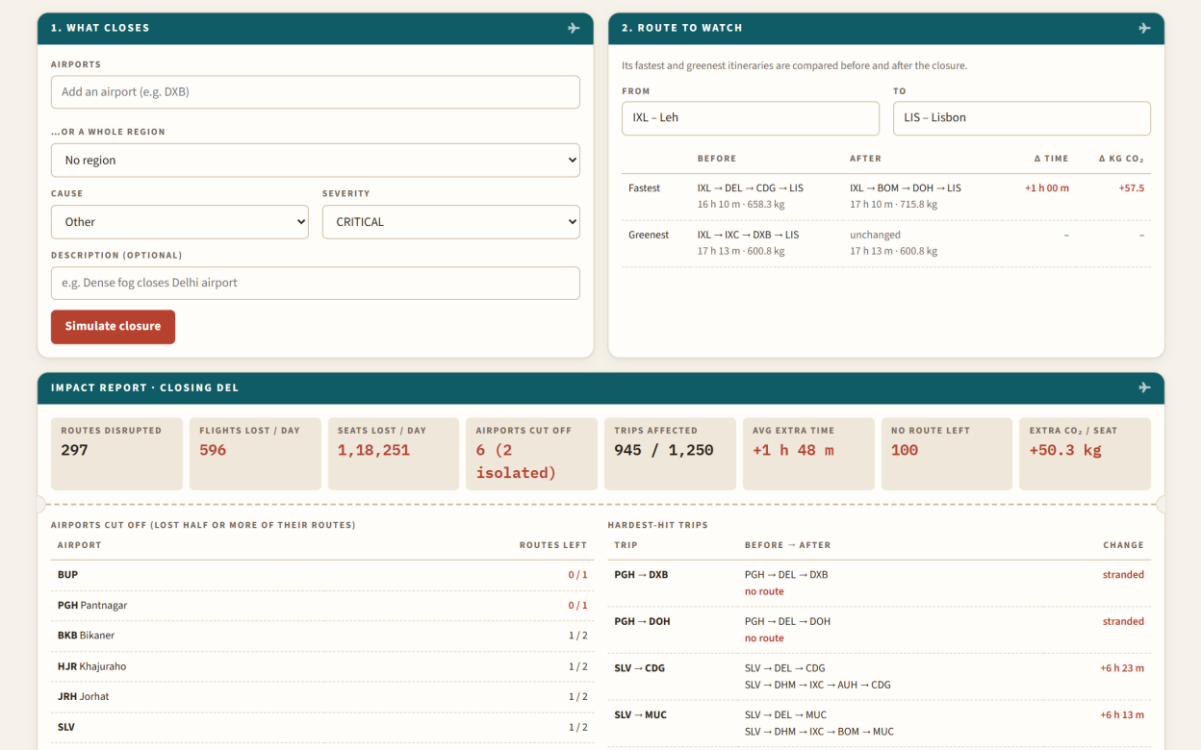


Figure 9: Disruption Simulator: closing Delhi re-routes Leh-Lisbon (+1 h, +57.5 kg CO2 per seat)

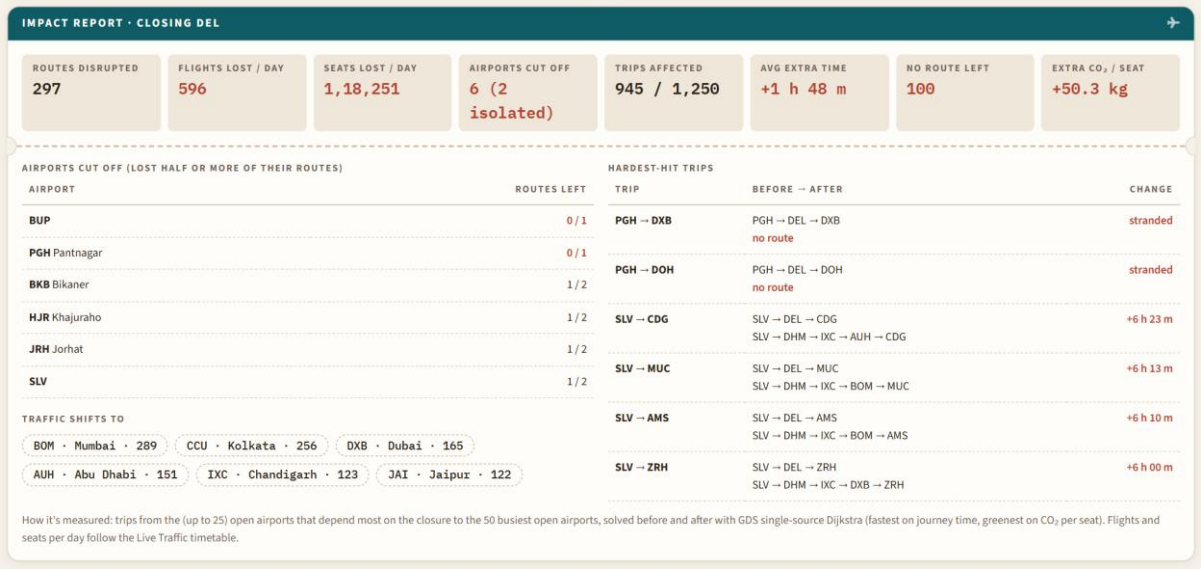


Figure 10: Sample report: disruption impact of closing Delhi

Table 34: Impact of closing Delhi and Dubai

Measure	**Close DEL**	**Close DXB**
Routes disrupted	297	517
Flights / seats lost per day	596 / 118,251	865 / 221,191
Airports cut off (isolated)	6 (2)	0 (0)
Sampled trips affected	945 of 1,250	363 of 1,250
Average extra time / trips left without a route	1 h 48 m / 100	1 h 21 m / 0
Extra CO2 per seat (greenest option)	+50.3 kg	+9.4 kg
Traffic shifts to	BOM, CCU, DXB, AUH	TBZ, IST, ADD, MED

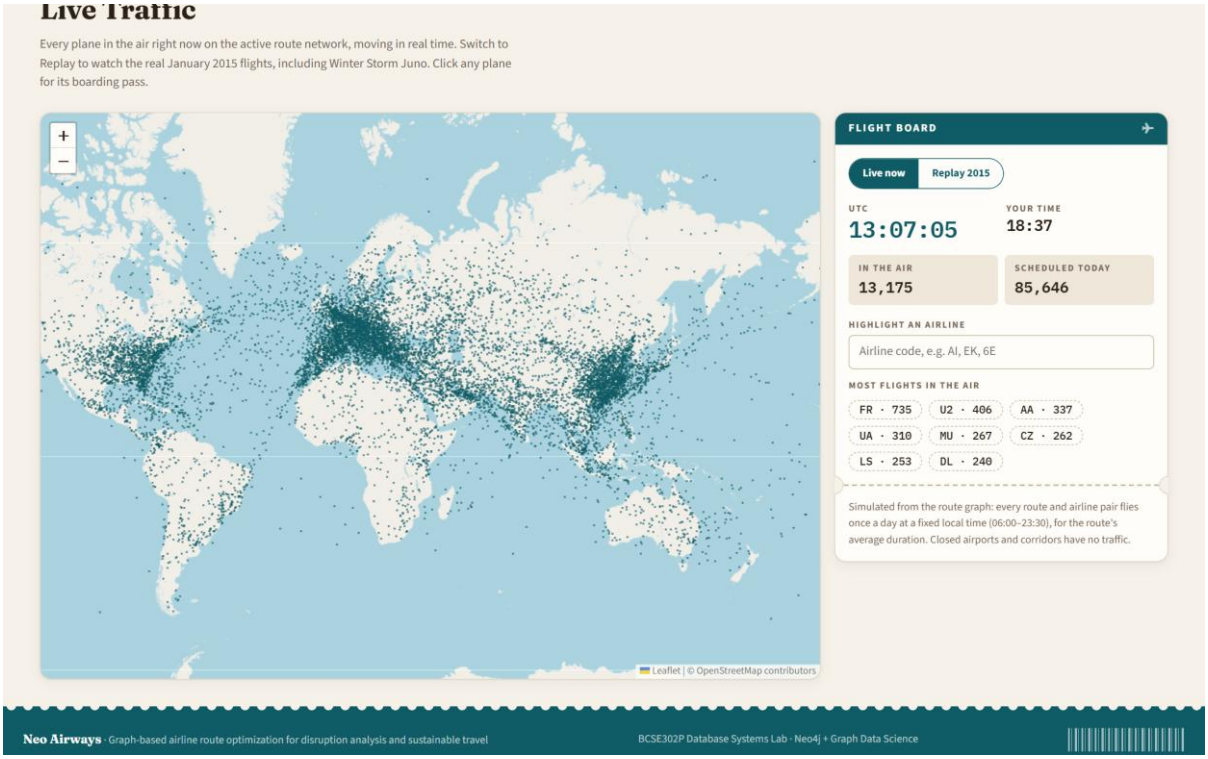


Figure 11: Live Traffic: simulated flights in the air at the current time

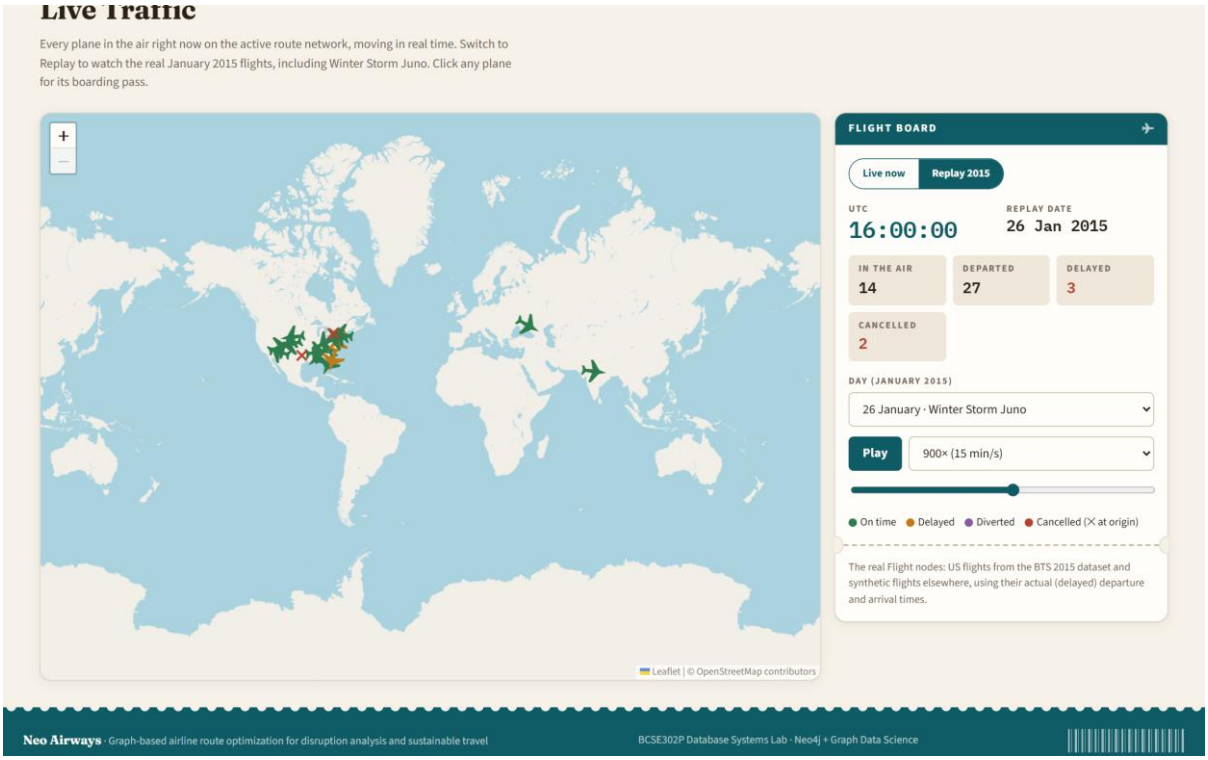


Figure 12: Live Traffic: replay of the real flights of 26 January 2015 (Winter Storm Juno)

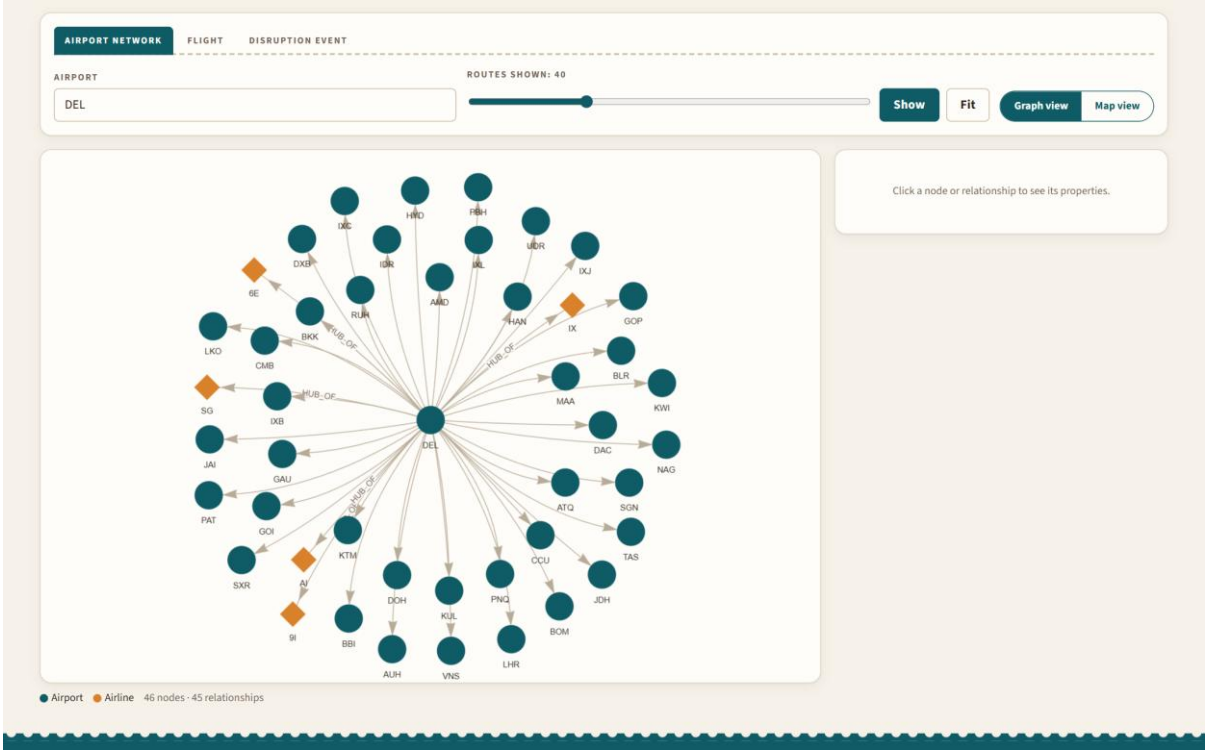


Figure 13: Graph Explorer: Delhi's busiest routes and its hub airlines

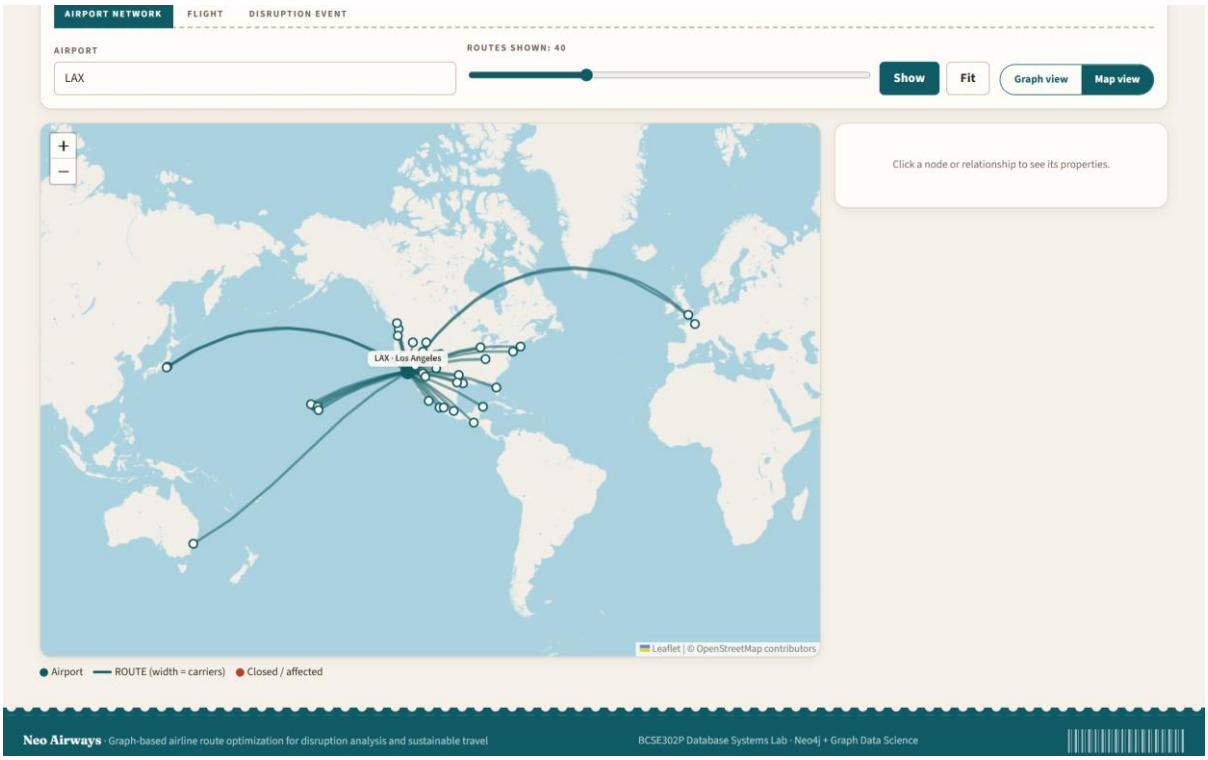


Figure 14: Graph Explorer, map view: the Los Angeles network

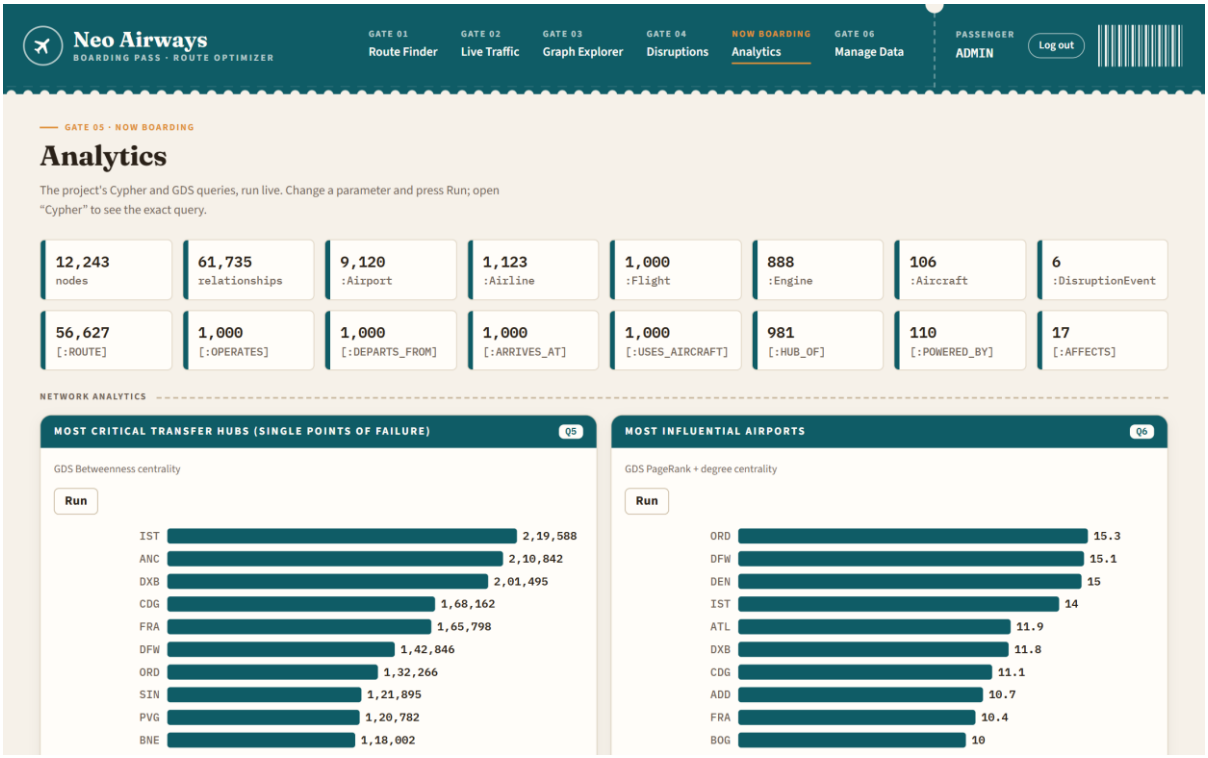


Figure 15: Analytics dashboard: database size and the first analytics cards

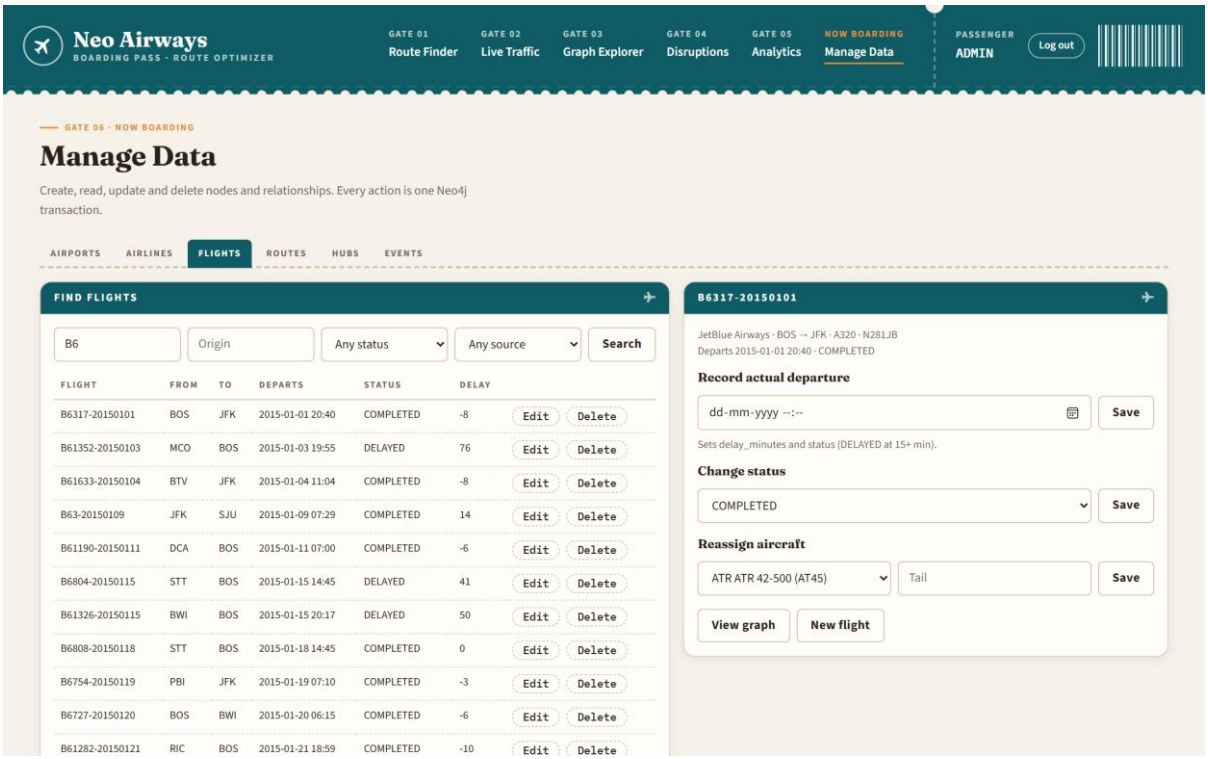


Figure 16: Data management (admin): flights with edit, departure, status and aircraft forms

7.2 Evaluation Against Objectives

Table 35: Evaluation against objectives

Objective	Evidence that it was met
O1 Graph model	6 labels, 8 relationship types, ER and graph schema (Figures 3-4), logical design (Section 4.5); multi-weight ROUTE edges
O2 Pipeline	Five idempotent seed scripts with dry runs; 12,243 nodes and 61,735 relationships loaded from six datasets; re-seeding routes in 38 s without duplicates
O3 CRUD, constraints, indexes	33 CRUD functions behind 46 endpoints; 12 constraints and 18 indexes; duplicate inserts rejected; the 57-check API test leaves the database unchanged
O4 Route optimisation	Fastest, greenest and shortest bookable itineraries in 0.14-0.41 s, alternatives, CO2 range and labels, unconstrained GDS optimum for comparison (Section 6.1.6)
O5 Disruptions	One-transaction closures with rollback on failure; impact report (Table above); exact restore (297 of 297 routes reactivated for DEL)
O6 Analytics	14 queries: Dijkstra, A*, Yen's, Betweenness, PageRank, aggregations, temporal (Juno) and spatial queries (Appendix D)
O7 Web application	Seven pages, interactive graph and maps, live traffic with about 11,000 moving aircraft, responsive from 375 px phones (Appendix F)

7.3 Comparison with the Existing System

Table 36: Comparison with existing systems

Aspect	Typical flight search / tracking tools	This project
Optimisation	Price and duration	Time, CO2 per seat and distance on the same graph, with explicit stop and time limits
Emissions	Per-result estimate	Per-leg model from engine data; CO2 range of all bookable choices
Disruptions	Not available to the public	What-if closures of airports or regions with an impact report, fully reversible
Network insight	None	Centrality, hub dependency, spatial alternatives
Data access	Closed or paid APIs	Open data, open source, reproducible
Live positions	Real ADS-B data	Simulated from the route graph (limitation)

7.4 Discussion of Findings

- **Most trips need a connection.** Only 0.43% of the 13.2 million possible airport pairs have a nonstop flight; in a random sample of 160 pairs among the 400 busiest airports, 76% needed a connection.
- **The choice of connection matters.** Across those pairs, the dirtiest bookable itinerary emits a median of 46% more CO2 per seat than the cleanest (up to 64%). The greenest itinerary differed from the fastest in 68% of pairs, saving a median of 10% for about 1.8 extra hours.
- **Nonstop flights are usually greenest when they exist.** Delhi-New York: the nonstop is fastest and greenest (778 kg per seat, 26% below the typical bookable option), because every extra stop adds a take-off and landing.
- **Model realism matters.** With maximum seat counts the model favoured routes through A380 hubs; typical seats and size-scaled fuel burn removed this bias and gave 55-88 g CO2 per seat-km for jets, close to published averages.

- **Fragility is not the same as size.** Closing Delhi strands 6 small airports and adds 1 h 48 m to affected trips, while closing Dubai, a larger hub, strands none because alternatives exist. Betweenness shows the same effect: Anchorage is the second most critical airport.
- **Real disruptions are visible in the data.** During Winter Storm Juno, 22 of 31 LaGuardia departures in the event window were cancelled (71%), detected automatically from cancellation spikes.

7.5 Limitations Observed

- Fuel burn values are synthetic within realistic ranges, so absolute CO2 numbers are estimates; comparisons between routes are more reliable than single values.
- The impact report samples 1,250 trips (25 dependent airports x 50 busy destinations) rather than all pairs, and its before/after paths are unconstrained Dijkstra paths.
- Live traffic is simulated with one daily departure per route and airline; real frequencies vary.
- Two application processes sharing the same projection name can interfere (observed once during testing); a single server is used for demonstrations.
- Backups are a documented manual procedure, not automated; the web pages need internet access for libraries and map tiles.

Chapter 8: Conclusion and Future Work

8.1 Summary of the Work

The project designed a property-graph model of the airline network from an ER model, loaded 12,243 nodes and 61,735 relationships from six public datasets with a reproducible pipeline, and built the Neo Airways web application on top of Neo4j. The system supports CRUD on nodes and relationships with constraints and indexes, finds bookable fastest, greenest and shortest itineraries, simulates and measures disruptions with ACID transactions, runs graph algorithms and analytics, and visualises the network as a graph, on maps and as live traffic.

8.2 Key Contributions

- A multi-criteria route graph whose CO2 weights are built from ICAO engine data, aircraft size and typical seating.
- An exact, fast search for bookable itineraries (stops and extra time bounded), with the CO2 range of all choices.
- A reversible disruption simulator with a measured network-wide impact report.
- A reproducible pipeline that resolves real-world data-quality problems across six datasets.
- An interactive, responsive web application with graph, map and live-traffic views.

8.3 Future Enhancements

- Use a published fuel-burn model (for example the methodology behind the ICAO Carbon Emissions Calculator or Google's Travel Impact Model [24]) and real seat configurations.
- Optional real-time positions from the OpenSky Network [27] alongside the simulated view.
- Airport-specific minimum connection times, real flight frequencies and fares for a cost optimisation.
- Multi-objective (Pareto) route sets instead of three single-objective answers.
- Community detection (Louvain) to find regional clusters, and a hub resilience ranking.
- Automated backups, offline copies of the web libraries, multiple user accounts, and deployment to Neo4j Aura.

8.4 Concluding Remarks

Treating the airline network as a graph made the central questions of this project (paths, weights, criticality and what-if closures) short to express and fast to answer, while Neo4j still provided the classical database guarantees of constraints, indexes and ACID transactions. The results show that the choice of connection has a large effect on emissions and that network fragility is not simply a matter of airport size, both of which a graph database is well placed to reveal.

Part C: References and Appendices

References

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377-387, 1970.

[2] R. Angles and C. Gutierrez, "Survey of graph database models," ACM Comput. Surv., vol. 40, no. 1, pp. 1-39, 2008.

[3] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[4] A. Pavlo and M. Aslett, "What's really new with NewSQL?," SIGMOD Rec., vol. 45, no. 2, pp. 45-55, 2016.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numer. Math., vol. 1, pp. 269-271, 1959.

[6] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100-107, 1968.

[7] J. Y. Yen, "Finding the k shortest loopless paths in a network," Manage. Sci., vol. 17, no. 11, pp. 712-716, 1971.

[8] U. Brandes, "A faster algorithm for betweenness centrality," J. Math. Sociol., vol. 25, no. 2, pp. 163-177, 2001.

[9] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," Comput. Netw. ISDN Syst., vol. 30, no. 1-7, pp. 107-117, 1998.

[10] R. Guimerà, S. Mossa, A. Turtschi, and L. A. N. Amaral, "The worldwide air transportation network: Anomalous centrality, community structure, and cities' global roles," Proc. Natl. Acad. Sci. USA, vol. 102, no. 22, pp. 7794-7799, 2005.

[11] B. Graver, K. Zhang, and D. Rutherford, "CO2 emissions from commercial aviation, 2018," International Council on Clean Transportation, Washington, DC, USA, 2019.

[12] European Union Aviation Safety Agency, ICAO Aircraft Engine Emissions Databank, version 32 (dataset).

[13] OurAirports, "Open data downloads." [Online]. Available: https://ourairports.com/data/

[14] OpenFlights, "Airport, airline and route data." [Online]. Available: https://openflights.org/data.html

[15] U.S. Department of Transportation, Bureau of Transportation Statistics, "2015 Flight Delays and Cancellations," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/usdot/flight-delays

[16] Neo4j, Inc., "The Neo4j Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/

[17] Neo4j, Inc., "The Neo4j Graph Data Science Library Manual." [Online]. Available: https://neo4j.com/docs/graph-data-science/

[18] Information technology - Database languages - GQL, ISO/IEC 39075:2024, 2024.

[19] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[20] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[21] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com/

[22] Leaflet: an open-source JavaScript library for interactive maps. [Online]. Available: https://leafletjs.com/

[23] vis-network documentation. [Online]. Available: https://visjs.github.io/vis-network/docs/network/

[24] Google, "Travel Impact Model." [Online]. Available: https://github.com/google/travel-impact-model

[25] International Air Transport Association, "Airline and airport code search." [Online]. Available: https://www.iata.org/en/publications/directories/code-search/

[26] Flightradar24. [Online]. Available: https://www.flightradar24.com/

[27] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up OpenSky: A large-scale ADS-B sensor network for research," in Proc. 13th Int. Symp. Inf. Process. Sensor Netw. (IPSN), 2014, pp. 83-94.

Appendix A: Complete DDL Scripts

Constraints and indexes, created by the seed scripts before data is loaded (`IF NOT EXISTS` makes them safe to re-run):

```
CREATE CONSTRAINT engine_uid IF NOT EXISTS FOR (e:Engine) REQUIRE e.engine_uid IS UNIQUE;
CREATE CONSTRAINT airport_id IF NOT EXISTS FOR (a:Airport) REQUIRE a.airport_id IS UNIQUE;
CREATE CONSTRAINT airline_id IF NOT EXISTS FOR (al:Airline) REQUIRE al.airline_id IS UNIQUE;
CREATE CONSTRAINT aircraft_type_id IF NOT EXISTS FOR (ac:Aircraft) REQUIRE ac.aircraft_type_id IS
UNIQUE;
CREATE CONSTRAINT flight_id IF NOT EXISTS FOR (f:Flight) REQUIRE f.flight_id IS UNIQUE;
CREATE CONSTRAINT event_id IF NOT EXISTS FOR (e:DisruptionEvent) REQUIRE e.event_id IS UNIQUE;
CREATE CONSTRAINT airport_name_exists IF NOT EXISTS FOR (a:Airport) REQUIRE a.name IS NOT NULL;
CREATE CONSTRAINT airport_location_type IF NOT EXISTS FOR (a:Airport) REQUIRE a.location IS ::
POINT;
CREATE CONSTRAINT flight_status_exists IF NOT EXISTS FOR (f:Flight) REQUIRE f.status IS NOT NULL;
CREATE CONSTRAINT flight_departure_type IF NOT EXISTS FOR (f:Flight) REQUIRE f.scheduled_departure
IS :: ZONED DATETIME;
CREATE CONSTRAINT route_weight_time_type IF NOT EXISTS FOR ()-[r:ROUTE]-() REQUIRE r.weight_time IS
:: FLOAT;
CREATE CONSTRAINT route_weight_carbon_type IF NOT EXISTS FOR ()-[r:ROUTE]-() REQUIRE
r.weight_carbon IS :: FLOAT;
CREATE INDEX airport_region IF NOT EXISTS FOR (a:Airport) ON (a.region);
CREATE INDEX airport_status IF NOT EXISTS FOR (a:Airport) ON (a.operational_status);
CREATE INDEX airport_country IF NOT EXISTS FOR (a:Airport) ON (a.country);
CREATE POINT INDEX airport_location IF NOT EXISTS FOR (a:Airport) ON (a.location);
CREATE FULLTEXT INDEX airport_search IF NOT EXISTS FOR (a:Airport) ON EACH [a.name, a.city,
a.airport_id];
CREATE INDEX flight_status IF NOT EXISTS FOR (f:Flight) ON (f.status);
CREATE INDEX flight_departure IF NOT EXISTS FOR (f:Flight) ON (f.scheduled_departure);
CREATE INDEX flight_data_source IF NOT EXISTS FOR (f:Flight) ON (f.data_source);
CREATE INDEX route_status IF NOT EXISTS FOR ()-[r:ROUTE]-() ON (r.corridor_status);
CREATE INDEX event_start IF NOT EXISTS FOR (e:DisruptionEvent) ON (e.start_time);
```

GDS projection of the active network (Section 5.5) and the memory settings in `neo4j.conf`:

```
server.memory.heap.initial_size=2g
server.memory.heap.max_size=4g
server.memory.pagecache.size=1g
```

Appendix B: Sample Data and DML Scripts

B.1 Sample records

Airport


```
{ "elevation_m": 11.9, "country": "IN", "daily_capacity": 500, "runway_count": 2, "operational_status":
"ACTIVE", "airport_id": "BOM", "city": "Mumbai", "timezone": "Asia/Kolkata", "name": "Chhatrapati Shivaji
International Airport", "location": "point(19.0887, 72.8679)", "region": "Asia", "is_hub": true }


{ "elevation_m": 236.8, "country": "IN", "daily_capacity": 500, "runway_count": 3, "operational_status":
"ACTIVE", "airport_id": "DEL", "city": "New Delhi", "timezone": "Asia/Kolkata", "name": "Indira Gandhi
International Airport", "location": "point(28.5665, 77.1031)", "region": "Asia", "is_hub": true }
```


Airline


```
{ "country": "India", "fleet_size": 259, "name": "IndiGo", "airline_id": "6E" }


{ "country": "India", "fleet_size": 74, "name": "Air India", "airline_id": "AI" }
```


Aircraft


```
{ "max_range_km": 6100, "avg_fuel_burn_kg_per_km": 2.8, "engine_count": 2, "typical_seats": 162,
"seating_capacity": 180, "aircraft_type_id": "A320", "model": "A320", "category": "Narrowbody",
"manufacturer": "Airbus" }


{ "max_range_km": 13649, "avg_fuel_burn_kg_per_km": 7.24, "engine_count": 2, "typical_seats": 358,
"seating_capacity": 550, "aircraft_type_id": "B77W", "model": "777-300ER", "category": "Widebody",
"manufacturer": "Boeing" }
```


Engine


```
{ "rated_thrust_kn": 513.9, "fuel_lto_cycle_kg": 1546, "bypass_ratio": 7.08, "engine_uid": "7GE099",
"engine_model": "GE90-115B", "manufacturer": "General Electric Company" }
```


Flight


```
{ "scheduled_arrival": "2015-01-01T16:14:00-06:00", "actual_arrival": "2015-01-01T16:39:00-06:00",
"distance_km": 1108.8, "flight_number": "DL1342", "delay_minutes": 24, "actual_departure": "2015-01-
01T15:19:00-05:00", "scheduled_departure": "2015-01-01T14:55:00-05:00", "flight_id": "DL1342-20150101",
"data_source": "BTS_2015", "status": "DELAYED", "duration_min": 139, "base_fare_usd": 160.88 }


{ "scheduled_arrival": "2015-01-28T23:35:00+00:00", "actual_arrival": "2015-01-28T22:52:00+00:00",
"distance_km": 7220, "flight_number": "VS2910", "delay_minutes": -12, "actual_departure": "2015-01-
28T18:58:00+05:30", "scheduled_departure": "2015-01-28T19:10:00+05:30", "flight_id": "VS2910-20150128",
"data_source": "SYNTHETIC", "status": "COMPLETED", "duration_min": 595, "base_fare_usd": 772 }
```


DisruptionEvent


```
{ "severity": "CRITICAL", "start_time": "2015-01-26T11:00:00+00:00", "event_id": "EVT-2015-0126-WEATHER",
"event_type": "WEATHER", "end_time": "2015-01-29T04:55:00+00:00", "description": "1903 weather
cancellations at BDL, BOS, DCA, EWR, FLL, JFK, LGA, MIA, PBI, PHL on 26-28 Jan 2015 (worst airport-day:
97% of departures cancelled)", "data_source": "BTS_2015" }
```


ROUTE (DEL -> BOM)


```
{ "corridor_status": "ACTIVE", "weight_cost": 163.3, "last_updated": "2026-10-25T09:48:19.518000+00:00",
"distance_km": 1133, "duration_estimated": false, "aircraft_types": [ "A319", "A320", "A321", "B738",
"B77W", "B788" ], "fuel_burn_estimate_kg": 6164.2, "weight_time": 140, "carriers": [ "6E", "AI", "IX",
"SG" ], "avg_speed_kmh": 485.6, "weight_carbon": 90.82, "fuel_burn_source": "EQUIPMENT_AIRLINE",
"co2_per_seat_kg": 90.82, "congestion_index": 0.57, "co2_emissions_kg": 19478.9, "avg_duration_min": 140 }
```


B.2 Loading statements (seed scripts)

MERGE_AIRLINES (seed.py):

```
UNWIND $rows AS row
MERGE (al:Airline {airline_id: row.IATA})
SET al.name = row.name,
    al.country = row.country,
    al.fleet_size = row.fleet_size
```

MERGE_AIRPORTS (seed.py):

```
UNWIND $rows AS row
MERGE (a:Airport {airport_id: row.iata_code})
SET a.name = row.name,
    a.city = row.municipality,
    a.country = row.iso_country,
    a.elevation_m = row.elevation_m,
    a.runway_count = row.runway_count,
    a.is_hub = row.is_hub,
    a.daily_capacity = row.daily_capacity,
    a.region = row.region,
    a.timezone = row.timezone,
    a.location = point({latitude: row.latitude_deg, longitude: row.longitude_deg}),
    // Keep any simulated disruption across re-seeds.
    a.operational_status = coalesce(a.operational_status, 'ACTIVE')
```

MERGE_ENGINES (seed.py):

```
UNWIND $rows AS row
MERGE (e:Engine {engine_uid: row.engine_uid})
SET e += row
```

MERGE_AIRCRAFT (seed2.py):

```
UNWIND $rows AS row
MERGE (ac:Aircraft {aircraft_type_id: row.aircraft_type_id})
SET ac += row
REMOVE ac.co2_emission_factor_kg_per_km
```

MERGE_HUBS (seed3.py):

```
UNWIND $rows AS row
MATCH (a:Airport {airport_id: row.airport_id})
MATCH (al:Airline {airline_id: row.airline_id})
MERGE (a)-[:HUB_OF]->(al)
```

MERGE_ROUTES (seed3.py):

```
UNWIND $rows AS row
MATCH (a:Airport {airport_id: row.src})
MATCH (b:Airport {airport_id: row.dst})
MERGE (a)-[r:ROUTE]->(b)
SET r += row.props,
    // Keep any simulated disruption across re-seeds.
    r.corridor_status = coalesce(r.corridor_status, 'ACTIVE'),
    r.last_updated = datetime()
```

MERGE_EVENTS (seed4.py):

```
UNWIND $rows AS row
MERGE (e:DisruptionEvent {event_id: row.event_id})
SET e += row.props,
    e.start_time = datetime(row.start_time),
    e.end_time = datetime(row.end_time)
WITH e, row
UNWIND row.airports AS code
MATCH (a:Airport {airport_id: code})
MERGE (e)-[:AFFECTS]->(a)
```

MERGE_FLIGHTS (seed4.py):

```
UNWIND $rows AS row
MATCH (al:Airline {airline_id: row.airline_id})
MATCH (o:Airport {airport_id: row.origin})
MATCH (d:Airport {airport_id: row.destination})
MATCH (ac:Aircraft {aircraft_type_id: row.aircraft_type_id})
MERGE (f:Flight {flight_id: row.flight_id})
SET f += row.props,
    f.scheduled_departure = datetime(row.scheduled_departure),
    f.scheduled_arrival = datetime(row.scheduled_arrival),
    f.actual_departure = datetime(row.actual_departure),
    f.actual_arrival = datetime(row.actual_arrival)
MERGE (al)-[:OPERATES]->(f)
MERGE (f)-[:DEPARTS_FROM]->(o)
MERGE (f)-[:ARRIVES_AT]->(d)
MERGE (f)-[u:USES_AIRCRAFT]->(ac)
SET u.tail_number = row.tail_number
```

MERGE_POWERED_BY (seed5.py):

```
UNWIND $rows AS row
MATCH (ac:Aircraft {aircraft_type_id: row.aircraft_type_id})
MATCH (e:Engine {engine_uid: row.engine_uid})
MERGE (ac)-[p:POWERED_BY]->(e)
SET p.is_default_engine = row.is_default_engine
```

B.3 CRUD statements (crud.py)

```
// Create a flight with its four relationships (one transaction)
MATCH (al:Airline {airline_id: $airline_id}), (o:Airport {airport_id: $origin}),
      (d:Airport {airport_id: $destination}), (ac:Aircraft {aircraft_type_id: $aircraft_type_id})
CREATE (f:Flight {flight_id: $flight_id})
SET f += $props, f.flight_number = $flight_number,
    f.scheduled_departure = datetime($scheduled_departure),
    f.scheduled_arrival = datetime($scheduled_arrival), f.status = coalesce($props.status,
'SCHEDULED')
CREATE (al)-[:OPERATES]->(f), (f)-[:DEPARTS_FROM]->(o), (f)-[:ARRIVES_AT]->(d),
       (f)-[:USES_AIRCRAFT {tail_number: $tail_number}]->(ac);

// Read an airport with its route count and hub airlines
MATCH (a:Airport {airport_id: $airport_id}) OPTIONAL MATCH (a)-[:HUB_OF]->(al:Airline)
RETURN a {.*} AS airport, COUNT { (a)-[:ROUTE]->() } AS routes_out, collect(al.airline_id) AS
hub_of;

// Update: record a departure; delay and status are derived
MATCH (f:Flight {flight_id: $flight_id})
SET f.actual_departure = datetime($actual_departure),
    f.delay_minutes = duration.inSeconds(f.scheduled_departure,
datetime($actual_departure)).seconds / 60
SET f.status = CASE WHEN f.delay_minutes >= 15 THEN 'DELAYED' ELSE 'COMPLETED' END;

// Update a relationship: disrupt a corridor
MATCH (:Airport {airport_id: $src})-[r:ROUTE]->(:Airport {airport_id: $dst})
SET r.corridor_status = 'DISRUPTED', r.last_updated = datetime();

// Replace a relationship: reassign the aircraft
MATCH (f:Flight {flight_id: $flight_id})-[old:USES_AIRCRAFT]->()
MATCH (ac:Aircraft {aircraft_type_id: $aircraft_type_id})
DELETE old CREATE (f)-[:USES_AIRCRAFT {tail_number: $tail_number}]->(ac);

// Delete a flight with its relationships
MATCH (f:Flight {flight_id: $flight_id}) DETACH DELETE f;
```

Appendix C: Stored Procedures, Functions and Triggers (Full Code)

Neo4j does not use SQL `CREATE PROCEDURE`, `CREATE FUNCTION` or `CREATE TRIGGER` statements. (Triggers are available only through the optional APOC plugin, `apoc.trigger.*`, which this project does not install.) Their roles are fulfilled by three mechanisms in this project:

1. **In-database procedures and functions** from Graph Data Science and the Neo4j system, invoked with Cypher's `CALL ... YIELD` clause (Section C.1).
2. **Application-layer transactional functions** in `queries.py` and `crud.py` that execute multi-step writes inside a single managed or explicit Bolt ACID transaction, playing the role of stored procedures (Section C.2), and reusable functions such as the itinerary search and the projection management (Section C.3).
3. **Trigger-equivalent reactive logic** in Cypher and Python that derives status flags, stamps timestamps, keeps derived weights consistent and invalidates the in-memory GDS projection whenever operational data changes (Section C.4).

C.1 In-database procedures and functions invoked

Table 37: In-database procedures and functions

Procedure / function	Kind	Purpose	Used in
`gds.graph.project`	GDS aggregation function	Builds the in-memory projection of ACTIVE airports and corridors with five weights	`queries.project_active_network`
`gds.graph.exists`, `gds.graph.drop`	GDS procedures	Check for and remove the projection	`ensure_projection`, `invalidate_projection`
`gds.shortestPath.dijkstra.stream`	GDS procedure	Weighted shortest path between two airports	Query 4, unconstrained optimum in the route finder
`gds.shortestPath.astar.stream`	GDS procedure	Shortest distance with a haversine heuristic	Query 13
`gds.shortestPath.yens.stream`	GDS procedure	k shortest loopless paths	Query 14
`gds.allShortestPaths.dijkstra.stream`	GDS procedure	Single-source shortest paths to all airports	Disruption impact report
`gds.betweenness.stream`, `gds.pageRank.stream`	GDS procedures	Centrality	Queries 5 and 6
`gds.util.asNode`, `gds.util.asNodes`	GDS functions	Map internal node ids to nodes	All GDS queries
`db.index.fulltext.queryNodes`	System procedure	Full-text airport search	`crud.search_airports`
`db.schema.visualization`, `db.schema.nodeTypeProperties`, `db.schema.relTypeProperties`	System procedures	Schema inspection	Report, Neo4j Browser
`point.distance`, `datetime`, `duration.inSeconds`	Built-in functions	Spatial distance; zoned times; delay calculation	Query 9, itinerary search, CRUD

C.2 Transactional functions (stored-procedure equivalents)

Each function below runs as one unit of work: either all of its statements commit or none do. `simulate_disruption` and `restore_disruption` use explicit transactions (`session.begin_transaction()` ... `tx.commit()`); the CRUD functions use managed transactions (`session.execute_write`), which the driver retries on transient errors.

queries.simulate_disruption

```
def simulate_disruption(driver, airports=(), region=None, severity="CRITICAL", event_type="OTHER",
                        description=None, event_id=None, impact=True):
    """Close airports (by code and/or a whole region) in one explicit transaction.


    The closure and its DisruptionEvent (linked by AFFECTS) commit together or
    not at all. With impact=True the network-wide damage is measured too (see
    disruption impact below) and its headline numbers stored on the event;
    otherwise the GDS projection is just dropped and rebuilt on next use.
    """
    event_id = event_id or f"EVT-WEB-{datetime.now(timezone.utc):%Y%m%d-%H%M%S}"
    sample = before = None
    if impact:
        with driver.session() as session:
            to_close = session.run(CLOSURE_SELECTION, airports=list(airports),
region=region).single()["codes"]
        sample = _impact_sample(driver, to_close)
        before = _impact_costs(driver, *sample) if sample[0] else None
    with driver.session() as session:
        with session.begin_transaction() as tx:
            codes = tx.run("""
                MATCH (a:Airport) WHERE a.airport_id IN $airports OR a.region = $region
                RETURN collect(a.airport_id) AS codes
                """, airports=list(airports), region=region).single()["codes"]
            closed = _close_airports(tx, codes, event_id)
            if not closed:
                raise ValueError("no ACTIVE airport matches that selection")
            what = region if region and not airports else ", ".join(closed[:5]) + ("..." if len(closed) >
5 else "")
            tx.run("""
                MERGE (e:DisruptionEvent {event_id: $event_id})
                SET e.event_type = $event_type, e.severity = $severity, e.data_source = 'MANUAL',
                    e.description = coalesce($description, 'Simulated closure of ' + $what),
                    e.start_time = datetime(), e.end_time = null
                WITH e
                MATCH (a:Airport) WHERE a.airport_id IN $closed
                MERGE (e)-[:AFFECTS]->(a)
                """, event_id=event_id, event_type=event_type, severity=severity,
                   description=description, what=what, closed=closed)
            tx.commit()
        disrupted = session.run("MATCH ()-[r:ROUTE {disrupted_by: $id}]->() RETURN count(r) AS n",
                                id=event_id).single()["n"]
    result = {"event_id": event_id, "closed_airports": closed, "disrupted_routes": disrupted}
    if not impact:
        invalidate_projection(driver)
        return result
    project_active_network(driver)   # needed now for the "after" measurements
    after = _impact_costs(driver, *sample) if before else None
    result["impact"] = _summarise_impact(driver, event_id, closed, before, after)
    return result
```

queries._close_airports

```
def _close_airports(tx, codes, event_id):
    """Soft-close ACTIVE airports and their ACTIVE corridors, tagging both with
    the event so restore_disruption() undoes exactly this closure."""
    closed = tx.run("""
        MATCH (a:Airport) WHERE a.airport_id IN $codes AND a.operational_status = 'ACTIVE'
        SET a.operational_status = 'CLOSED', a.disrupted_by = $event_id
        RETURN collect(a.airport_id) AS codes
        """, codes=codes, event_id=event_id).single()["codes"]
    tx.run("""
        MATCH (a:Airport)-[r:ROUTE]-() WHERE a.airport_id IN $codes AND r.corridor_status = 'ACTIVE'
        SET r.corridor_status = 'DISRUPTED', r.disrupted_by = $event_id
        """, codes=closed, event_id=event_id)
    return closed
```

queries.restore_disruption

```
def restore_disruption(driver, event_id):
    """Reopen exactly the airports and corridors an event closed, and end the event."""
    with driver.session() as session:
        with session.begin_transaction() as tx:
            reopened = tx.run("""
                MATCH (a:Airport {disrupted_by: $id})
                SET a.operational_status = 'ACTIVE' REMOVE a.disrupted_by
                RETURN count(a) AS n
                """, id=event_id).single()["n"]
            restored = tx.run("""
                MATCH ()-[r:ROUTE {disrupted_by: $id}]->()
                SET r.corridor_status = 'ACTIVE' REMOVE r.disrupted_by
```

```
                RETURN count(r) AS n
                """, id=event_id).single()["n"]
            tx.run("MATCH (e:DisruptionEvent {event_id: $id}) SET e.end_time = datetime()", id=event_id)
            tx.commit()
    invalidate_projection(driver)
    return {"event_id": event_id, "reopened_airports": reopened, "restored_routes": restored}
```

crud.create_flight

```
def create_flight(driver, flight_id, flight_number, airline_id, origin, destination,
                  aircraft_type_id, scheduled_departure, scheduled_arrival,
                  tail_number=None, **props):
    """Creates the Flight and its OPERATES / DEPARTS_FROM / ARRIVES_AT /
    USES_AIRCRAFT relationships in one transaction."""
    return _one(_write(driver, """
        MATCH (al:Airline {airline_id: $airline_id})
        MATCH (o:Airport {airport_id: $origin})
        MATCH (d:Airport {airport_id: $destination})
        MATCH (ac:Aircraft {aircraft_type_id: $aircraft_type_id})
        CREATE (f:Flight {flight_id: $flight_id})
        SET f += $props, f.flight_number = $flight_number,
            f.scheduled_departure = datetime($scheduled_departure),
            f.scheduled_arrival = datetime($scheduled_arrival),
            f.status = coalesce($props.status, 'SCHEDULED')
        CREATE (al)-[:OPERATES]->(f), (f)-[:DEPARTS_FROM]->(o), (f)-[:ARRIVES_AT]->(d),
               (f)-[:USES_AIRCRAFT {tail_number: $tail_number}]->(ac)
        RETURN f {.*} AS flight
        """, flight_id=flight_id, flight_number=flight_number, airline_id=airline_id,
        origin=origin, destination=destination, aircraft_type_id=aircraft_type_id,
        scheduled_departure=scheduled_departure, scheduled_arrival=scheduled_arrival,
        tail_number=tail_number, props=props))
```

crud.reassign_aircraft

```
def reassign_aircraft(driver, flight_id, aircraft_type_id, tail_number=None):
    """Replaces the USES_AIRCRAFT relationship."""
    return _one(_write(driver, """
        MATCH (f:Flight {flight_id: $flight_id})-[old:USES_AIRCRAFT]->()
        MATCH (ac:Aircraft {aircraft_type_id: $aircraft_type_id})
        DELETE old
        CREATE (f)-[u:USES_AIRCRAFT {tail_number: $tail_number}]->(ac)
        RETURN f.flight_id AS flight_id, ac.model AS aircraft, u.tail_number AS tail_number
        """, flight_id=flight_id, aircraft_type_id=aircraft_type_id, tail_number=tail_number))
```

crud.create_event

```
def create_event(driver, event_id, event_type, severity, start_time, airports,
                 description, end_time=None, data_source="MANUAL"):
    """Creates the event and an AFFECTS relationship to each airport."""
    return _one(_write(driver, """
        CREATE (e:DisruptionEvent {event_id: $event_id, event_type: $event_type,
                severity: $severity, description: $description, data_source: $data_source,
                start_time: datetime($start_time)})
        SET e.end_time = datetime($end_time)
        WITH e
        UNWIND $airports AS code
        MATCH (a:Airport {airport_id: code})
        CREATE (e)-[:AFFECTS]->(a)
        RETURN e {.*} AS event, collect(a.airport_id) AS affects
        """, event_id=event_id, event_type=event_type, severity=severity, start_time=start_time,
        end_time=end_time, airports=airports, description=description, data_source=data_source))
```

crud._write / crud._read

```
def _write(driver, query, **params):
    with driver.session() as session:
        return plain(session.execute_write(lambda tx: tx.run(query, **params).data()))



def _read(driver, query, **params):
    with driver.session() as session:
        return plain(session.execute_read(lambda tx: tx.run(query, **params).data()))
```

C.3 Reusable functions

The itinerary search is the project's main read-only function: it runs the UNION ALL Cypher search (Section 6.1.6), applies the extra-time rule and adds the unconstrained GDS optimum. The projection functions maintain the virtual view `active_network`.

queries.extra_time_allowed

```
def extra_time_allowed(fastest_min):
    """How much longer than the fastest a greener/shorter itinerary may be:
    75% of the fastest journey under 15 h, 50% from 15 h, never over 10 h."""
    share = 0.75 if fastest_min < 15 * 60 else 0.5
    return min(MAX_EXTRA_MIN, round(share * fastest_min))
```

queries.find_routes

```
def find_routes(driver, src, dst, k=3, max_stops=2, max_extra_min=None, graph=ACTIVE_GRAPH):
    """Bookable fastest, greenest and shortest itineraries, plus k alternatives.


    Only itineraries with at most `max_stops` stops count, and the greenest and
    shortest may be at most `max_extra_min` longer than the fastest (default:
    extra_time_allowed()). Alternatives are the next-fastest bookable ones.
    "unconstrained" is GDS Dijkstra's greenest path over the whole network,
    reported when it differs (usually more stops or a far longer trip).
    """
    max_stops = max(0, min(2, max_stops))
    cypher = ITINERARIES.format(patterns="\n  UNION ALL\n  ".join(ITINERARY_PATTERNS[:max_stops + 1]))
    options, alternatives = [], []
    with _projection_lock:
        ensure_projection(driver, graph)
        with driver.session() as session:
            found = session.run(cypher, src=src, dst=dst, detour=DETOUR,
                                connection=CONNECTION_MIN).data()
            limits = {"max_stops": max_stops, "candidates": len(found)}
            if found:
                fastest = min(found, key=lambda c: (c["journey"], c["legs"], c["co2"]))
                extra = max_extra_min if max_extra_min is not None else
extra_time_allowed(fastest["journey"])
                within = [c for c in found if c["journey"] <= fastest["journey"] + extra]
                greenest = min(within, key=lambda c: (c["co2"], c["journey"]))
                shortest = min(within, key=lambda c: (c["km"], c["journey"]))
                co2 = [c["co2"] for c in within]
                limits.update(fastest_min=fastest["journey"], max_extra_min=extra,
within_limit=len(within),
                              co2_min=round(min(co2), 1), co2_typical=round(statistics.median(co2), 1),
                              co2_max=round(max(co2), 1))
                stops = f"≤{max_stops} stop{'s' if max_stops != 1 else ''}"
                for key, label, pick, how in [
                        ("fastest", "Fastest", fastest, f"least journey time, {stops}"),
                        ("greenest", "Greenest", greenest, "least CO₂ per seat within the time limit"),
                        ("shortest", "Shortest distance", shortest, "least km within the time limit")]:
                    options.append(_option(session, key, label, how, pick["path"]))
                shown = {tuple(o["path"]) for o in options}
                for c in sorted(within, key=lambda c: (c["journey"], c["co2"])):
                    if len(alternatives) == k:
                        break
                    if tuple(c["path"]) not in shown:
                        shown.add(tuple(c["path"]))
                        alternatives.append(_option(session, f"alt{len(alternatives) + 1}",
                                                    f"Alternative {len(alternatives) + 1}",
                                                    "next fastest bookable itinerary", c["path"]))

            # Reference: the unconstrained carbon optimum from GDS Dijkstra.
            rows = session.run(PATH_DIJKSTRA, src=src, dst=dst, graph=graph,
weight="weight_carbon").data()
            unconstrained = None
            if rows and rows[0]["path"] and (not options or rows[0]["path"] != options[1]["path"]):
                unconstrained = _option(session, "unconstrained", "Unconstrained optimum",
                                        "GDS Dijkstra on weight_carbon, no stop or time limit",
rows[0]["path"])
            # How each itinerary compares with the typical (median) bookable one.
            for o in options + alternatives + ([unconstrained] if unconstrained else []):
                if limits.get("co2_typical"):
                    o["vs_typical_pct"] = round(100 * (o["co2_per_seat_kg"] - limits["co2_typical"]) /
limits["co2_typical"])
            codes = sorted({c for o in options + alternatives + [unconstrained or {"path": []}] for c in
o["path"]}
                           | {src, dst})
```

```
            stops = {r["code"]: r for r in plain(session.run(PATH_STOPS, codes=codes).data())}
    return {"src": src, "dst": dst, "airports": stops, "options": options, "alternatives": alternatives,
            "unconstrained": unconstrained, "limits": limits}
```

queries.project_active_network / ensure_projection / invalidate_projection

```
def project_active_network(driver, graph=ACTIVE_GRAPH):
    """(Re)build the in-memory GDS graph of the currently active network."""
    with _projection_lock, driver.session() as session:
        session.run("CALL gds.graph.drop($graph, false) YIELD graphName RETURN graphName",
graph=graph).consume()
        return session.run(PROJECT_ACTIVE, graph=graph, connection_min=float(CONNECTION_MIN)).data()[0]



def ensure_projection(driver, graph=ACTIVE_GRAPH):
    """Project the active network only if it isn't in memory yet."""
    with _projection_lock:
        with driver.session() as session:
            exists = session.run("CALL gds.graph.exists($graph) YIELD exists RETURN exists",
                                 graph=graph).single()["exists"]
        if not exists:
            return project_active_network(driver, graph)



def invalidate_projection(driver, graph=ACTIVE_GRAPH):
    """Drop the projection after a network change; the next GDS query rebuilds
    it once (ensure_projection), instead of re-projecting after every write."""
    with _projection_lock, driver.session() as session:
        session.run("CALL gds.graph.drop($graph, false) YIELD graphName RETURN graphName",
graph=graph).consume()
```

C.4 Trigger-equivalent logic

Table 38: Trigger-equivalent reactions

Event ("trigger")	Reaction	Where
A departure time is recorded	`delay_minutes` is derived from the scheduled and actual times; `status` becomes DELAYED (15 min or more) or COMPLETED	`crud.record_departure` (Cypher)
A route is updated	`last_updated` is set to `datetime()`	`crud.update_route` (Cypher)
A route's duration or CO2 is edited	`weight_time` / `weight_carbon` are updated to match	`app.update_route`
Any write that changes the ACTIVE network (airport, route or status change, disruption, restore)	The GDS projection is dropped and the live-traffic timetable invalidated; both rebuild on next use	`app.network_changed`, `traffic.invalidate`
A manual event is deleted	Anything it still holds closed is restored first	`app.delete_event`
Re-seeding the data	Existing `operational_status` and `corridor_status` values are kept (`coalesce`), so a simulated disruption survives	`MERGE_AIRPORTS`, `MERGE_ROUTES` (Appendix B)

crud.record_departure (derives delay_minutes and status)

```
def record_departure(driver, flight_id, actual_departure, delayed_at_min=15):
    """Sets actual_departure and derives delay_minutes and status from it."""
    return _one(_write(driver, """
        MATCH (f:Flight {flight_id: $flight_id})
        SET f.actual_departure = datetime($actual_departure),
            f.delay_minutes = duration.inSeconds(f.scheduled_departure,
datetime($actual_departure)).seconds / 60
        SET f.status = CASE WHEN f.delay_minutes >= $delayed_at_min THEN 'DELAYED' ELSE 'COMPLETED' END
        RETURN f {.*} AS flight
        """, flight_id=flight_id, actual_departure=actual_departure, delayed_at_min=delayed_at_min))
```

crud.update_route (stamps last_updated)

```
def update_route(driver, src, dst, **props):
    return _one(_write(driver, """
        MATCH (:Airport {airport_id: $src})-[r:ROUTE]->(:Airport {airport_id: $dst})
        SET r += $props, r.last_updated = datetime()
        RETURN r {.*} AS route
        """, src=src, dst=dst, props=props))
```

app.network_changed (after every network write)

```
def network_changed(driver):
    """After a write that changes the ACTIVE network, drop the GDS projection
    and the live-traffic timetable; both rebuild lazily on next use. The write
    has already committed, so a failure here is logged, not reported."""
    try:
        queries.invalidate_projection(driver)
    except Neo4jError as e:
        log.warning("could not drop the GDS projection: %s", e.message)
    traffic.invalidate()
```

app.update_route (keeps weights in step with their source values)

```
@app.put("/api/routes/{src}/{dst}", dependencies=admin)
def update_route(src: str, dst: str, body: RouteUpdate, driver=Depends(db)):
    props = body.model_dump(exclude_unset=True)
    # Keep the pathfinding weights in step with the values they mirror.
    if "avg_duration_min" in props:
        props["weight_time"] = float(props["avg_duration_min"])
    if "co2_per_seat_kg" in props:
        props["weight_carbon"] = float(props["co2_per_seat_kg"])
    result = found(crud.update_route(driver, src.upper(), dst.upper(), **props), f"Route {src}->{dst}")
    network_changed(driver)
    return result
```

app.set_corridor_status (fires network_changed)

```
@app.put("/api/routes/{src}/{dst}/status", dependencies=admin)
def set_corridor_status(src: str, dst: str, body: CorridorIn, driver=Depends(db)):
    result = found(crud.set_corridor_status(driver, src.upper(), dst.upper(), body.status), f"Route {src}-
>{dst}")
    network_changed(driver)
    return result
```

app.delete_event (restores before deleting)

```
@app.delete("/api/events/{event_id}", dependencies=admin)
def delete_event(event_id: str, driver=Depends(db)):
    event = found(crud.get_event(driver, event_id), f"Event {event_id}")
    if event["event"].get("data_source") != "MANUAL":
        raise HTTPException(409, "Only manually created events can be deleted")
    queries.restore_disruption(driver, event_id)  # reopen anything it still holds closed
    traffic.invalidate()
    return deleted(crud.delete_event(driver, event_id), f"Event {event_id}")
```

traffic.invalidate (rebuild the live timetable)

```
def invalidate(driver=None):
    """Call after anything changes the ACTIVE network (closures, route edits);
    with a driver, the new timetable is rebuilt in the background straight away."""
    global _timetable
    with _lock:
        _timetable = None
    if driver is not None:
        warm(driver)
```

Appendix D: Complete Query Set with Outputs

All analytical queries from `queries.py`, run against the live database (first rows shown; GDS queries run on the projection `active_network`).

Query 1: Direct routes from Delhi, longest first

Feature: Graph traversal (1 hop) + relationship properties (10 rows, 8 ms)

```
MATCH (d:Airport {airport_id: $src})-[r:ROUTE {corridor_status: 'ACTIVE'}]->(x:Airport)
RETURN x.airport_id AS dest, x.city AS city, x.country AS country,
       r.distance_km AS km, r.avg_duration_min AS minutes,
       r.co2_per_seat_kg AS co2_per_seat_kg, r.carriers AS carriers
ORDER BY km DESC LIMIT 10
```

dest	city	country	km	minutes	co2_per_seat_kg	carriers
SFO	San Francisco	US	12,404	930	875.82	AI
EWR	Newark	US	11,787	1,013	778.84	AI, UA
JFK	New York	US	11,777	1,010	778.20	AA, AI
YYZ	Toronto	CA	11,661	995	916.14	AC, AI
YUL	Montréal	CA	11,295	978	897.28	AC
YVR	Vancouver	CA	11,161	920	790.42	AI

Query 2: Destinations reachable from Bengaluru with at most one stop

Feature: Variable-length traversal [:ROUTE*1..2] (6 rows, 25 ms)

```
MATCH p = (b:Airport {airport_id: $src})-[:ROUTE*1..2]->(x:Airport)
WHERE x <> b AND all(r IN relationships(p) WHERE r.corridor_status = 'ACTIVE')
WITH x, min(length(p)) AS flights
RETURN x.region AS region,
       sum(CASE flights WHEN 1 THEN 1 ELSE 0 END) AS nonstop,
       sum(CASE flights WHEN 2 THEN 1 ELSE 0 END) AS one_stop,
       count(*) AS total
ORDER BY total DESC
```

region	nonstop	one_stop	total
Asia	90	375	465
Europe	5	227	232
Africa	2	129	131
North America	0	68	68
Oceania	1	61	62
South America	0	12	12

Query 3: Fewest-connections itinerary from Leh to Lisbon

Feature: Cypher shortestPath (unweighted hops) (1 rows, 0 ms)

```
MATCH (a:Airport {airport_id: $src}), (b:Airport {airport_id: $dst})
MATCH p = shortestPath((a)-[:ROUTE*..8]->(b))
WHERE all(r IN relationships(p) WHERE r.corridor_status = 'ACTIVE')
RETURN [n IN nodes(p) | n.airport_id + ' (' + n.city + ')'] AS itinerary,
       length(p) AS flights,
       round(reduce(km = 0.0, r IN relationships(p) | km + r.distance_km)) AS total_km
```

itinerary	flights	total_km
IXL (Leh) -> BOM (Mumbai) -> AMS (Amsterdam) -> LIS (Lisbon)	3	10,444

Query 4: Fastest vs greenest route (GDS Dijkstra on two weights)

Feature: GDS shortest path: gds.shortestPath.dijkstra (2 rows, 17 ms)

```
MATCH (s:Airport {airport_id: $src}), (t:Airport {airport_id: $dst})
UNWIND ['weight_time', 'weight_carbon'] AS weight
CALL gds.shortestPath.dijkstra.stream($graph, {
  sourceNode: s, targetNode: t, relationshipWeightProperty: weight
}) YIELD nodeIds
WITH weight, gds.util.asNodes(nodeIds) AS stops
UNWIND range(0, size(stops) - 2) AS i
WITH weight, stops, stops[i] AS a, stops[i + 1] AS b
MATCH (a)-[r:ROUTE]->(b)
WITH weight, stops, sum(r.avg_duration_min) AS flight_minutes,
     sum(r.co2_per_seat_kg) AS co2_per_seat_kg, sum(r.distance_km) AS km
RETURN CASE weight WHEN 'weight_time' THEN 'FASTEST' ELSE 'GREENEST' END AS optimised_for,
       [n IN stops | n.airport_id] AS path, flight_minutes,
       round(co2_per_seat_kg, 1) AS co2_per_seat_kg, round(km) AS km
```

optimised_for	path	flight_minutes	co2_per_seat_kg	km
FASTEST	DEL -> TAS -> JFK	970	899.10	11,778
GREENEST	DEL -> JFK	1,010	778.20	11,777

Query 5: Most critical transfer hubs (single points of failure)

Feature: GDS Betweenness centrality (10 rows, 293 ms)

```
CALL gds.betweenness.stream($graph, {samplingSize: 1000, samplingSeed: 42})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS a, score
RETURN a.airport_id AS airport, a.city AS city, a.country AS country, round(score) AS betweenness
ORDER BY betweenness DESC LIMIT 10
```

airport	city	country	betweenness
IST	Arnavutköy, Istanbul	TR	219,588
ANC	Anchorage	US	210,842
DXB	Dubai	AE	201,495
CDG	Paris	FR	168,162
FRA	Frankfurt am Main	DE	165,798
DFW	Dallas-Fort Worth	US	142,846

Query 6: Most influential airports

Feature: GDS PageRank + degree centrality (10 rows, 51 ms)

```
CALL gds.pageRank.stream($graph, {maxIterations: 20, dampingFactor: 0.85})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS a, score
ORDER BY score DESC LIMIT 10
RETURN a.airport_id AS airport, a.city AS city, round(score, 2) AS pagerank,
       COUNT { (a)-[:ROUTE {corridor_status: 'ACTIVE'}]->() } AS out_degree,
       COUNT { (a)<-[:ROUTE {corridor_status: 'ACTIVE'}]-() } AS in_degree
ORDER BY pagerank DESC
```

airport	city	pagerank	out_degree	in_degree
ORD	Chicago	15.30	282	279
DFW	Dallas-Fort Worth	15.13	270	270
DEN	Denver	15.02	235	235
IST	Arnavutköy, Istanbul	14.04	308	306
ATL	Atlanta	11.92	245	244
DXB	Dubai	11.83	259	258

Query 7: Airline on-time performance

Feature: Aggregation across Airline-OPERATES-Flight (12 rows, 6 ms)

```
MATCH (al:Airline)-[:OPERATES]->(f:Flight)
WITH al, f.data_source AS source, count(f) AS flights,
     avg(f.delay_minutes) AS avg_delay,
```

```
     sum(CASE f.status WHEN 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
     sum(CASE f.status WHEN 'DELAYED' THEN 1 ELSE 0 END) AS delayed
WHERE flights >= $min_flights
RETURN al.airline_id AS airline, al.name AS name, source, flights,
       round(avg_delay, 1) AS avg_delay_min,
       round(100.0 * delayed / flights, 1) AS delayed_pct,
       round(100.0 * cancelled / flights, 1) AS cancelled_pct
ORDER BY cancelled_pct DESC, delayed_pct DESC LIMIT 12
```

airline	name	source	flights	avg_delay_min	delayed_pct	cancelled_pct
B6	JetBlue Airways	BTS_2015	65	17.10	20	36.90
UA	United Airlines	BTS_2015	54	12.90	20.40	29.60
MQ	American Eagle Airlines	BTS_2015	39	29	30.80	28.20
US	US Airways	BTS_2015	64	4.40	12.50	28.10
DL	Delta Air Lines	BTS_2015	100	10.10	13	21
AI	Air India	SYNTHETIC	15	4.80	6.70	20

Query 8: Impact of Winter Storm Juno on departures

Feature: Multi-hop pattern + temporal filter + aggregation (10 rows, 12 ms)

```
MATCH (e:DisruptionEvent {event_id: $event})-[:AFFECTS]->(a:Airport)<-[:DEPARTS_FROM]-(f:Flight)
WHERE f.scheduled_departure >= e.start_time AND f.scheduled_departure <= e.end_time
RETURN a.airport_id AS airport, count(f) AS departures_in_window,
       sum(CASE f.status WHEN 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
       round(avg(f.delay_minutes), 1) AS avg_delay_min_of_operated
ORDER BY cancelled DESC
```

airport	departures_in_window	cancelled	avg_delay_min_of_operated
LGA	31	22	21
BOS	27	16	18.10
JFK	26	13	27.80
EWR	25	11	15
PHL	18	7	3.80
DCA	4	3	-2

Query 9: Airports within 300 km of Chennai

Feature: Spatial query (point.distance, point index) (9 rows, 5 ms)

```
MATCH (c:Airport {airport_id: $center})
WITH c.location AS here
MATCH (a:Airport)
WHERE point.distance(a.location, here) < $metres AND a.location <> here
RETURN a.airport_id AS airport, a.name AS name, a.city AS city,
       round(point.distance(a.location, here) / 1000) AS km,
       COUNT { (a)-[:ROUTE]->() } AS routes
ORDER BY km LIMIT 10
```

airport	name	city	km	routes
TIR	Tirupati Airport	Tirupati	99	7
PNY	Pondicherry Airport	Pondicherry	120	2
NVY	Neyveli Airport	Neyveli	168	0
CDP	Kadapa Airport	Kadapa	227	3
SXV	Salem Airport	Salem	265	4
BLR	Kempegowda International Airport	Bangalore	268	98

Query 10: Carbon efficiency by aircraft category

Feature: Aggregation with the CO2 constant applied in Cypher (6 rows, 0 ms)

```
MATCH (ac:Aircraft) WHERE ac.typical_seats > 0
```

```
WITH ac.category AS category, count(*) AS types,
     avg(ac.avg_fuel_burn_kg_per_km * 3.16 / ac.typical_seats * 1000) AS g_co2_per_seat_km
RETURN category, types, round(g_co2_per_seat_km, 1) AS g_co2_per_seat_km
ORDER BY g_co2_per_seat_km
```

category	types	g_co2_per_seat_km
Narrowbody	33	64.80
Turboprop	16	72.60
Widebody	27	74.70
Regional	25	96.80
Supersonic	1	499.60
Private Jet	2	551.40

Query 11: Engines of the aircraft flying Delhi-Mumbai, most efficient first

Feature: 4-label traversal: ROUTE -> Aircraft -POWERED_BY-> Engine (10 rows, 11 ms)

```
MATCH (:Airport {airport_id: $src})-[r:ROUTE]->(:Airport {airport_id: $dst})
UNWIND r.aircraft_types AS type_id
MATCH (ac:Aircraft {aircraft_type_id: type_id})-[p:POWERED_BY]->(e:Engine)
RETURN ac.model AS aircraft, e.engine_model AS engine, e.manufacturer AS maker,
       round(e.bypass_ratio, 2) AS bypass_ratio, round(e.fuel_lto_cycle_kg) AS lto_fuel_kg,
       p.is_default_engine AS default_engine
ORDER BY bypass_ratio DESC
```

aircraft	engine	maker	bypass_ratio	lto_fuel_kg	default_engine
787-8	Trent 1000-AE3	Rolls-Royce plc	9.14	884	false
787-8	GEnx-1B64	General Electric Company	9	796	true
777-300ER	GE90-115B	General Electric Company	6.96	1,449	true
A319	CFM56-5B5/3	CFM International	6	343	true
A320	CFM56-5B4	CFM International	5.90	421	true
A321	CFM56-5B3/3	CFM International	5.40	478	true

Query 12: Hub dependency: airlines whose network relies most on one hub

Feature: Pattern comprehension + aggregation over HUB_OF and ROUTE (10 rows, 73 ms)

```
MATCH ()-[r:ROUTE]->()
UNWIND r.carriers AS carrier
WITH carrier, count(*) AS all_routes
WHERE all_routes >= $min_routes
MATCH (hub:Airport)-[:HUB_OF]->(al:Airline {airline_id: carrier})
WITH al, all_routes, hub,
     COUNT { (hub)-[h:ROUTE]->() WHERE al.airline_id IN h.carriers } AS hub_routes
WITH al, all_routes, collect(hub.airport_id) AS hubs, sum(hub_routes) AS via_hubs
RETURN al.airline_id AS airline, al.name AS name, all_routes, hubs,
       round(100.0 * via_hubs / all_routes, 1) AS pct_routes_from_hubs
ORDER BY pct_routes_from_hubs DESC LIMIT 10
```

airline	name	all_routes	hubs	pct_routes_from_hubs
VA	Virgin Australia	163	ADL, BNE, MEL, PER, SYD	54
W9	Wizz Air UK	123	LGW, LTN	53.70
F3	Flyadeal	118	DMM, JED, MED, RUH	53.40
J9	Jazeera Airways	179	DMM, KWI	52
3O	Air Arabia Maroc	108	CMN, FEZ, NDR, RBA, TNG	50.90
OU	Croatia Airlines	118	DBV, SPU, ZAG	50.80

Query 13: Shortest-distance route from Delhi to New York

Feature: GDS A* (haversine heuristic on Airport coordinates) (1 rows, 17 ms)

```
MATCH (s:Airport {airport_id: $src}), (t:Airport {airport_id: $dst})
```

```
CALL gds.shortestPath.astar.stream($graph, {
  sourceNode: s, targetNode: t, relationshipWeightProperty: 'distance_km',
  latitudeProperty: 'latitude', longitudeProperty: 'longitude'
}) YIELD nodeIds, totalCost
RETURN [n IN gds.util.asNodes(nodeIds) | n.airport_id] AS path,
       size(nodeIds) - 1 AS flights, round(totalCost) AS km
```

path	flights	km
DEL -> JFK	1	11,777

Query 14: Top 3 fastest itineraries from Delhi to New York

Feature: GDS Yen's k-shortest paths (route recommendation) (3 rows, 30 ms)

```
MATCH (s:Airport {airport_id: $src}), (t:Airport {airport_id: $dst})
CALL gds.shortestPath.yens.stream($graph, {
  sourceNode: s, targetNode: t, k: $k, relationshipWeightProperty: 'weight_trip'
}) YIELD index, nodeIds, totalCost
RETURN index + 1 AS rank, [n IN gds.util.asNodes(nodeIds) | n.airport_id] AS path,
       size(nodeIds) - 1 AS flights, round(totalCost - $connection_min) AS journey_minutes
ORDER BY rank
```

rank	path	flights	journey_minutes
1	DEL -> JFK	1	1,010
2	DEL -> TAS -> JFK	2	1,060
3	DEL -> MUC -> JFK	2	1,123

Disruption impact: trip costs with GDS single-source Dijkstra

```
MATCH (t:Airport) WHERE t.airport_id IN $targets
WITH collect(id(t)) AS tids
UNWIND $origins AS code
MATCH (s:Airport {airport_id: code})
// An airport left with no active route isn't in the projection: its trips have no route at all.
WHERE s.operational_status = 'ACTIVE'
  AND EXISTS { (s)-[:ROUTE {corridor_status: 'ACTIVE'}]-(:Airport {operational_status: 'ACTIVE'}) }
CALL gds.allShortestPaths.dijkstra.stream($graph, {sourceNode: s, relationshipWeightProperty: $weight})
YIELD targetNode, totalCost, nodeIds
WITH code, targetNode, totalCost, nodeIds, tids WHERE targetNode IN tids
RETURN code AS src, gds.util.asNode(targetNode).airport_id AS dst, totalCost AS cost,
       CASE WHEN $paths THEN [n IN gds.util.asNodes(nodeIds) | n.airport_id] END AS path
```

Disruption impact: daily flights and seats lost

```
MATCH ()-[r:ROUTE {disrupted_by: $id}]->()
CALL (r) {
  UNWIND r.aircraft_types AS type_id
  MATCH (ac:Aircraft {aircraft_type_id: type_id}) WHERE ac.typical_seats > 0
  RETURN avg(ac.typical_seats) AS seats
}
RETURN sum(size(r.carriers)) AS flights, round(sum(size(r.carriers) * coalesce(seats, 0))) AS seats
```

Appendix E: Full-Size ER Diagram and Relational Schema

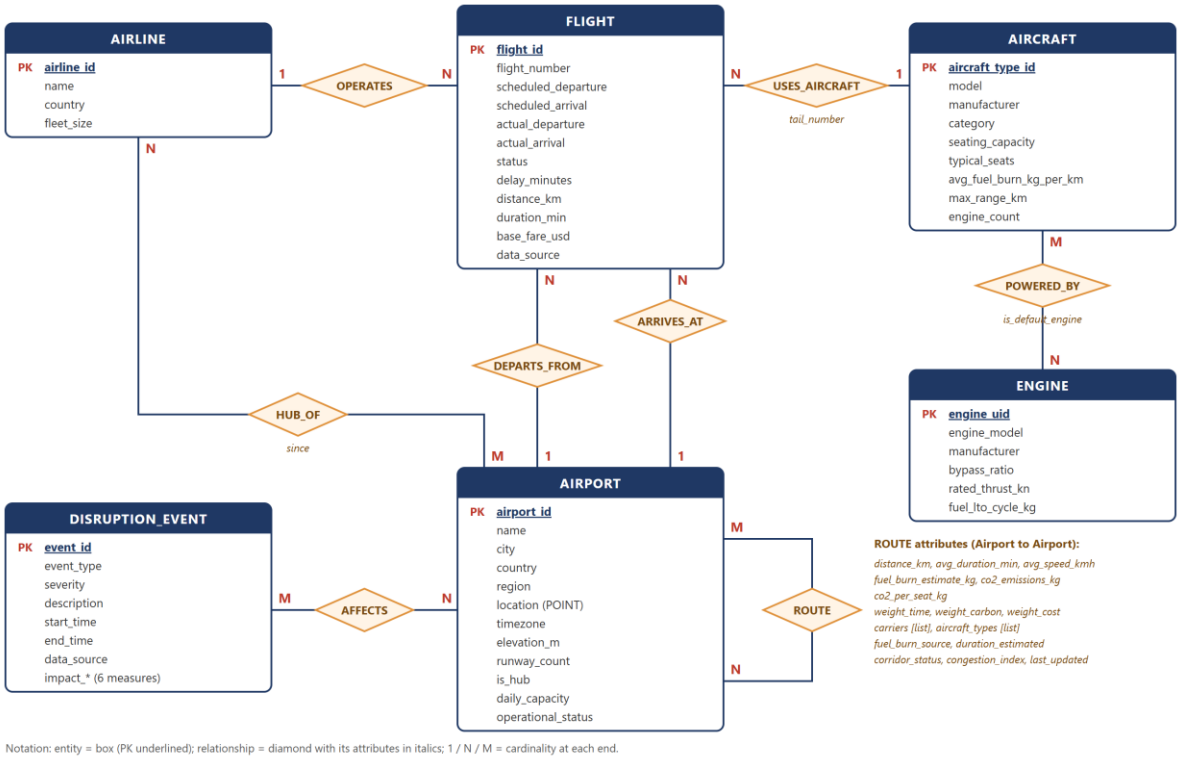


Figure 17: Entity-relationship diagram (full size)

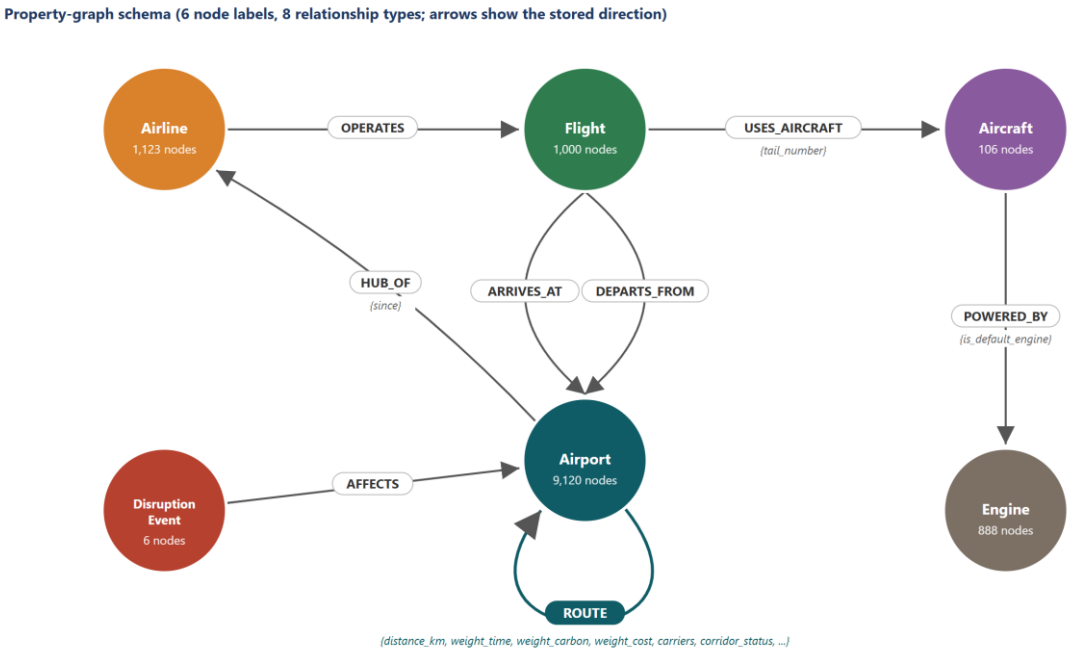


Figure 18: Property-graph schema

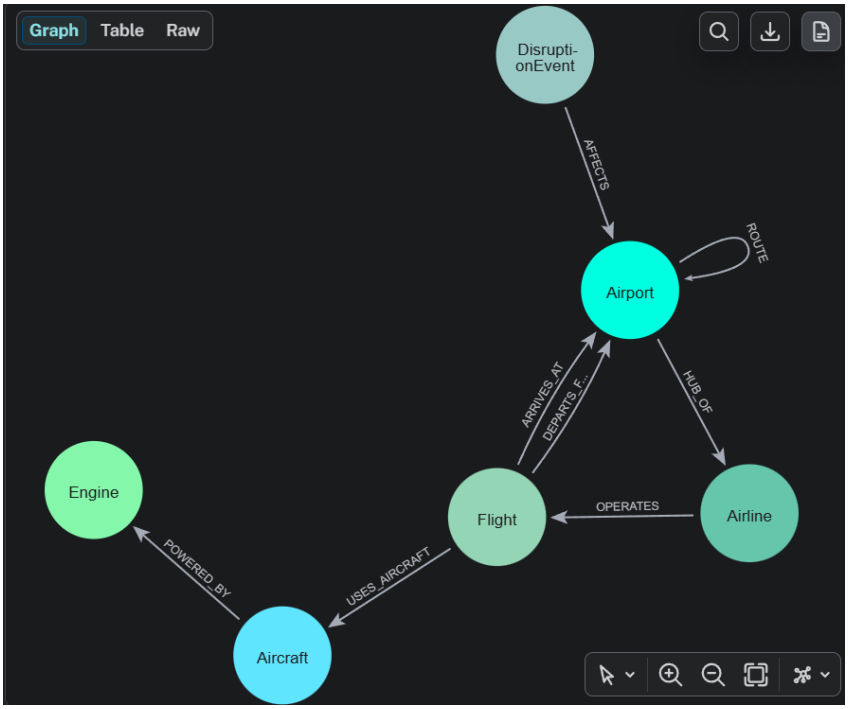


Figure 19: Schema generated by Neo4j (`CALL db.schema.visualization()`)

Equivalent relational schema (for comparison; PK underlined in the ER diagram, FK = foreign key):

```
AIRPORT(airport_id PK, name, city, country, region, latitude, longitude, timezone, elevation_m,
        runway_count, is_hub, daily_capacity, operational_status)
AIRLINE(airline_id PK, name, country, fleet_size)
AIRCRAFT(aircraft_type_id PK, model, manufacturer, category, seating_capacity, typical_seats,
         avg_fuel_burn_kg_per_km, max_range_km, engine_count)
ENGINE(engine_uid PK, engine_model, manufacturer, bypass_ratio, rated_thrust_kn, fuel_lto_cycle_kg)
FLIGHT(flight_id PK, flight_number, airline_id FK, origin FK->AIRPORT, destination FK->AIRPORT,
       aircraft_type_id FK, tail_number, scheduled_departure, scheduled_arrival, actual_departure,
       actual_arrival, status, delay_minutes, distance_km, duration_min, base_fare_usd, data_source)
DISRUPTION_EVENT(event_id PK, event_type, severity, description, start_time, end_time, data_source, ...)
ROUTE(src FK->AIRPORT, dst FK->AIRPORT, PK(src, dst), distance_km, avg_duration_min, ..., corridor_status)
ROUTE_CARRIER(src, dst, airline_id FK, PK(src, dst, airline_id))       -- list property in the graph
ROUTE_AIRCRAFT(src, dst, aircraft_type_id FK, PK(src, dst, aircraft_type_id))
HUB_OF(airport_id FK, airline_id FK, since, PK(airport_id, airline_id))
POWERED_BY(aircraft_type_id FK, engine_uid FK, is_default_engine, PK(aircraft_type_id, engine_uid))
AFFECTS(event_id FK, airport_id FK, PK(event_id, airport_id))
```

Appendix F: Application Screenshots

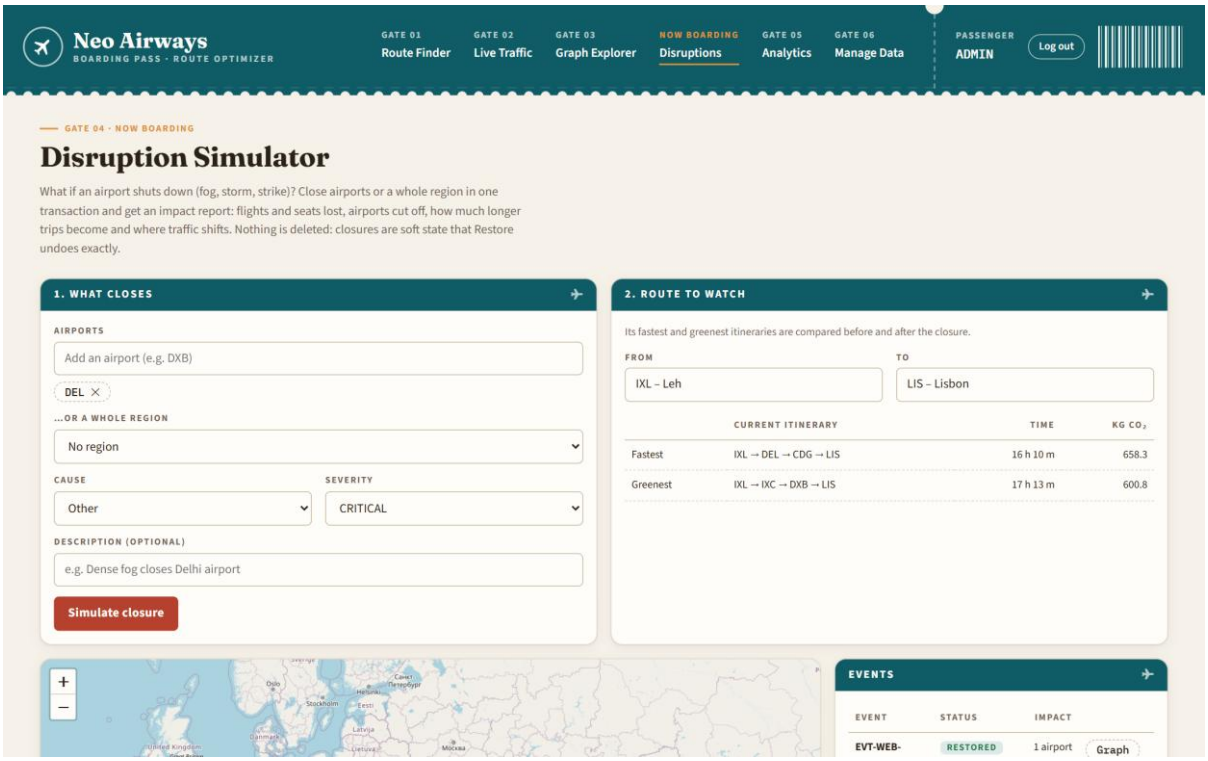


Figure 20: Disruption Simulator before a closure

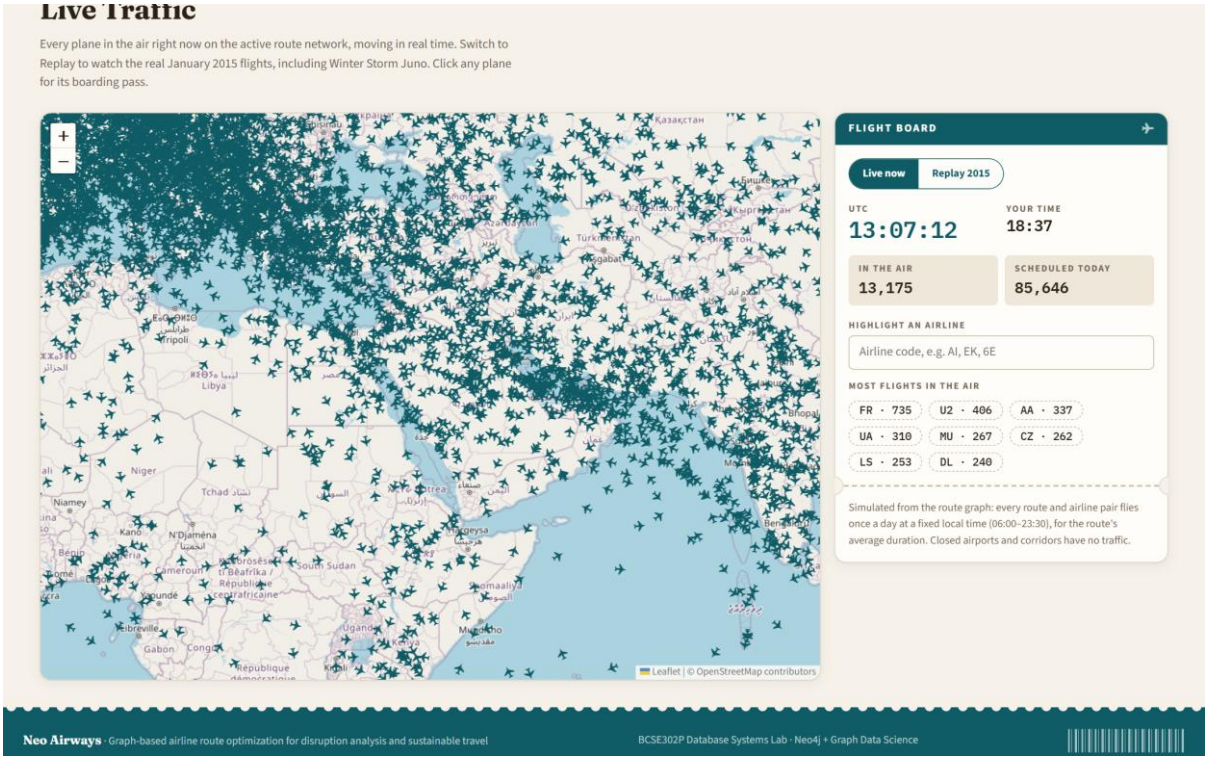


Figure 21: Live Traffic zoomed in: aircraft icons point along their routes

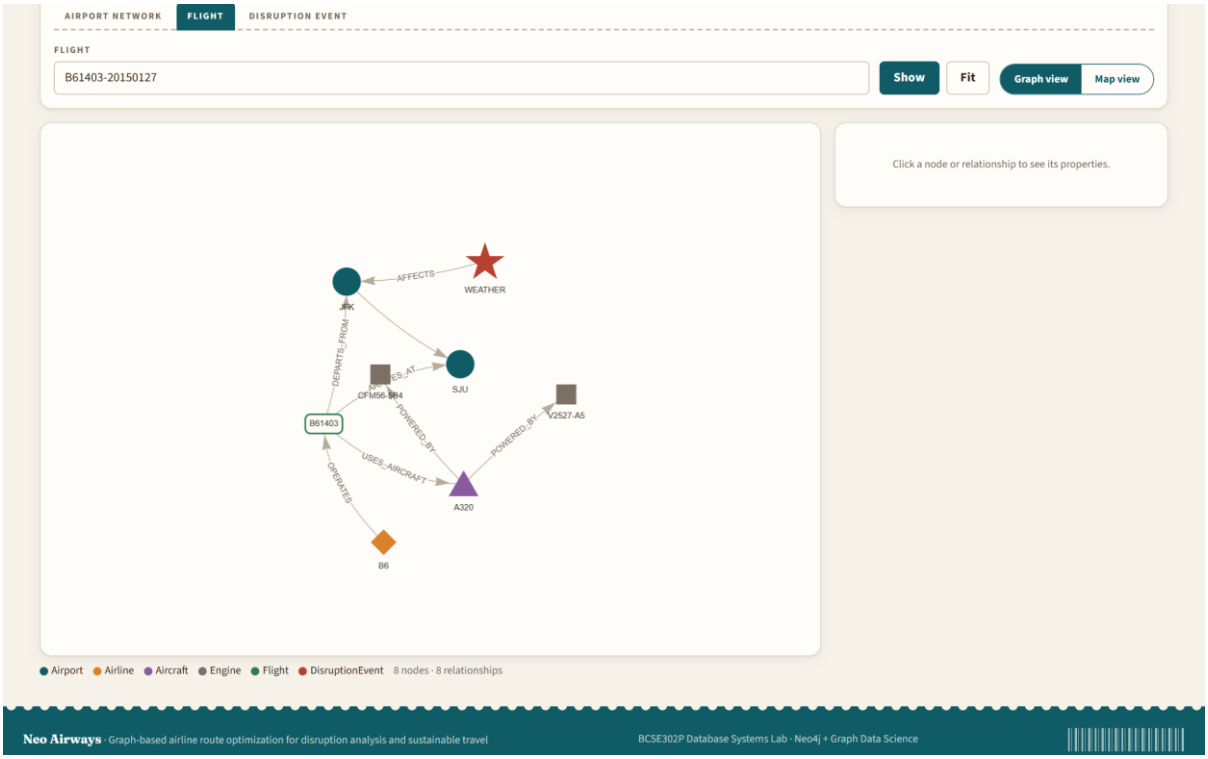


Figure 22: Graph Explorer: a cancelled Juno flight with its airline, airports, aircraft, engine and the event

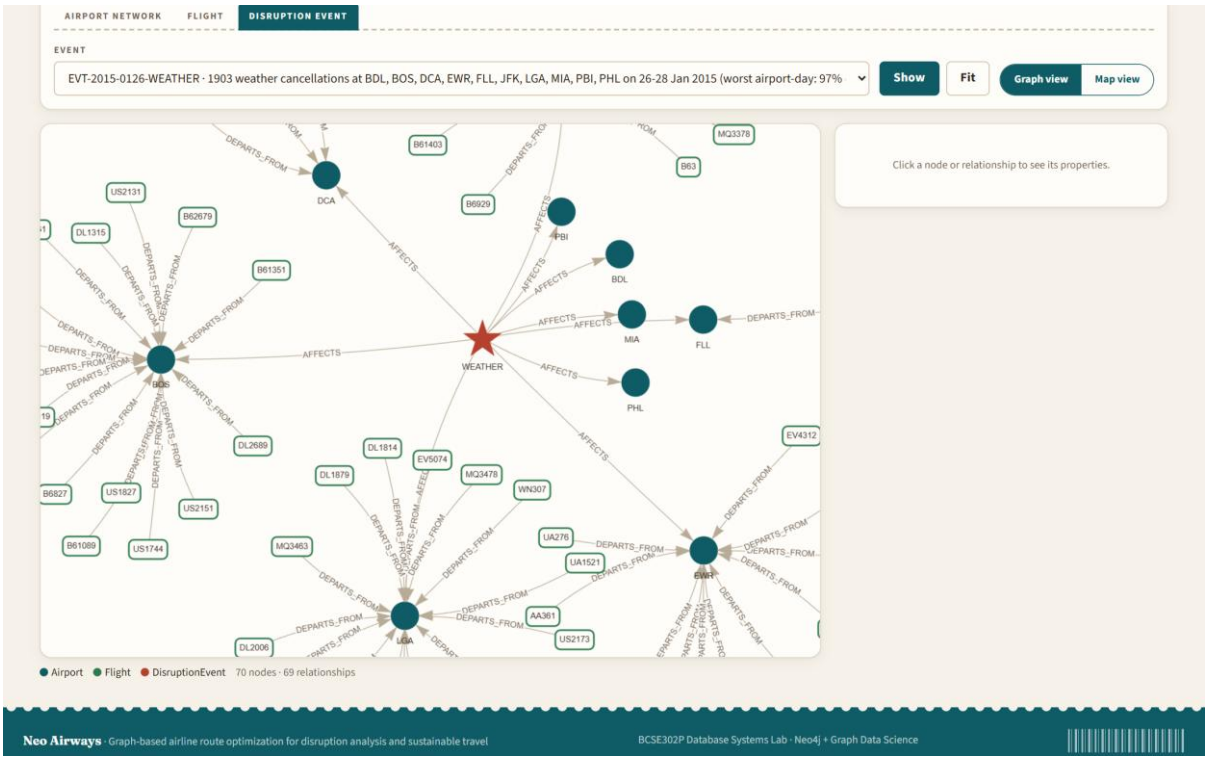


Figure 23: Graph Explorer: Winter Storm Juno with affected airports and cancelled flights

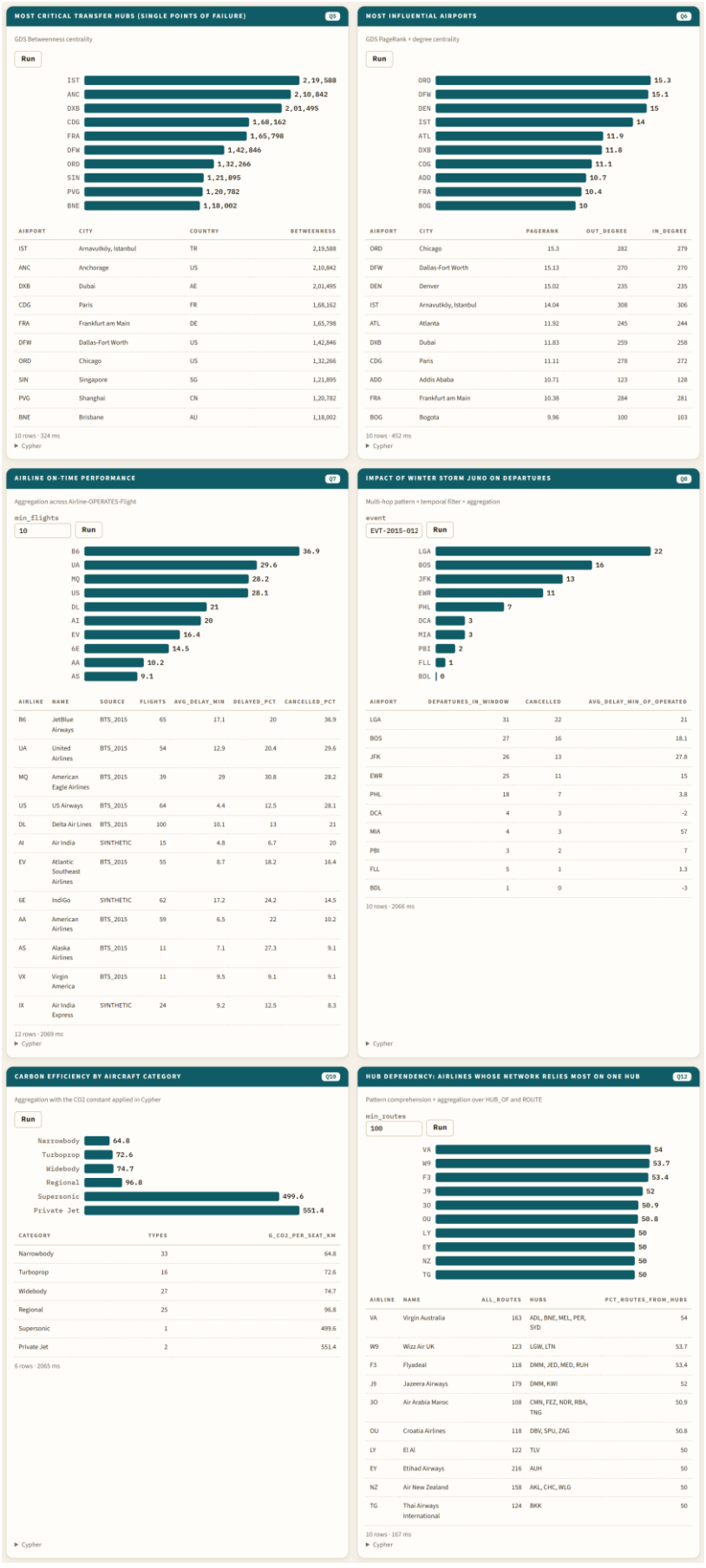


Figure 24: Analytics cards with bar charts and the Cypher behind each query

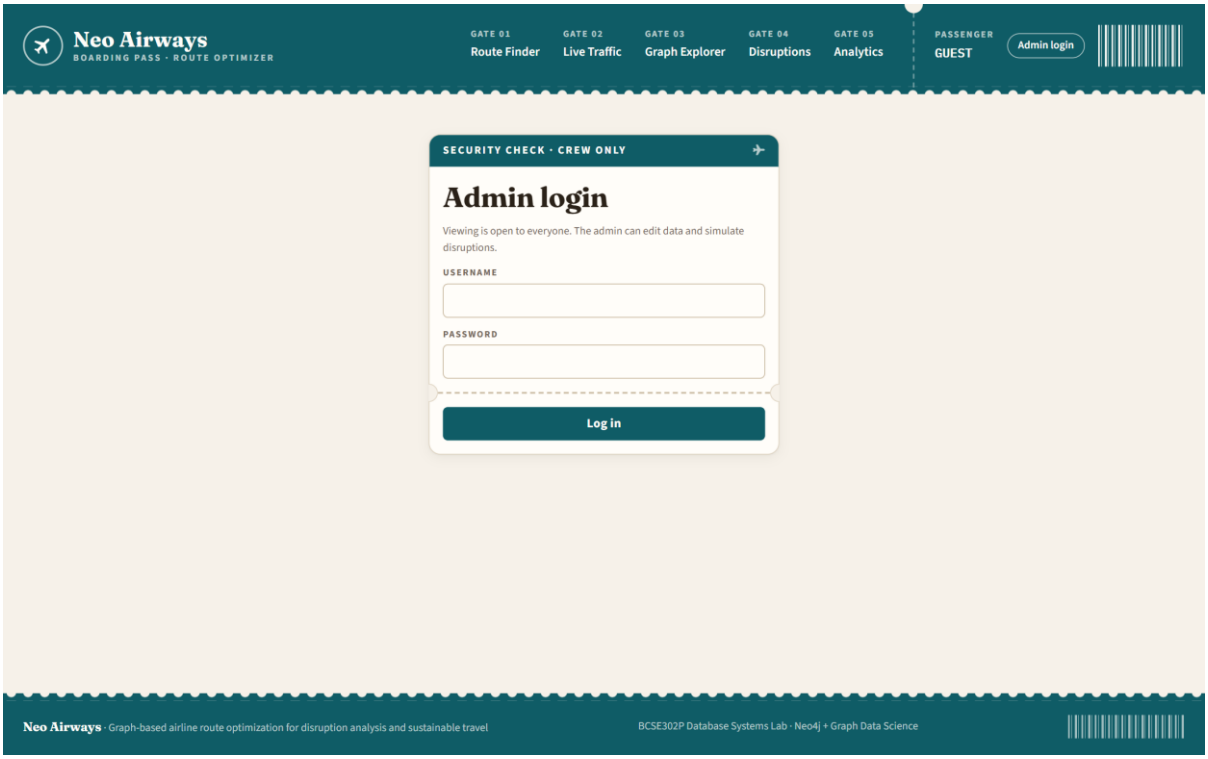


Figure 25: Admin login



Figure 26: Automatic API documentation generated by FastAPI (/docs)

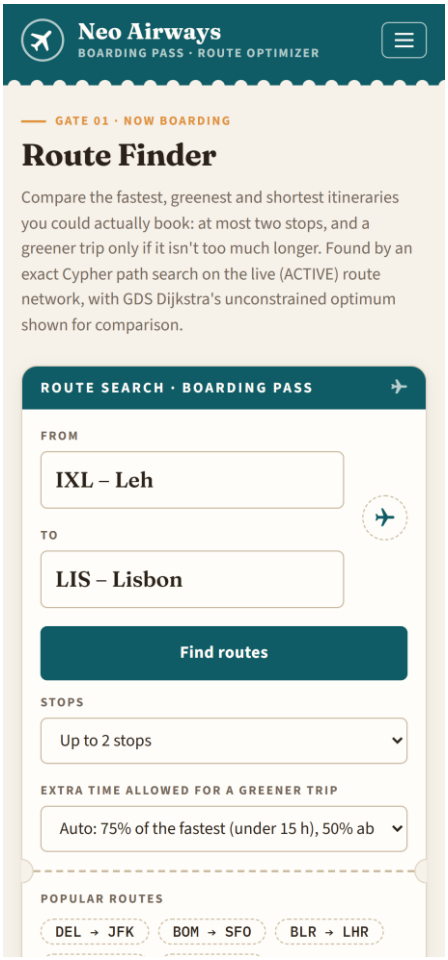


Figure 27: Phone layout (390 px): route search



Figure 28: Phone layout: boarding-pass ticket

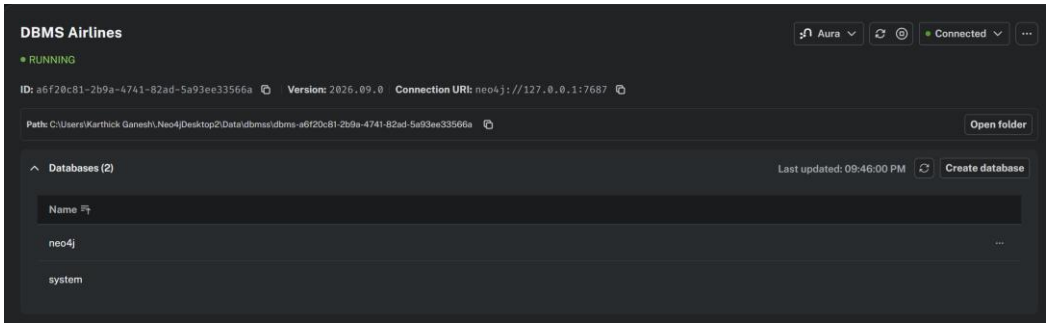


Figure 29: Neo4j Desktop: the DBMS running with the Graph Data Science plugin

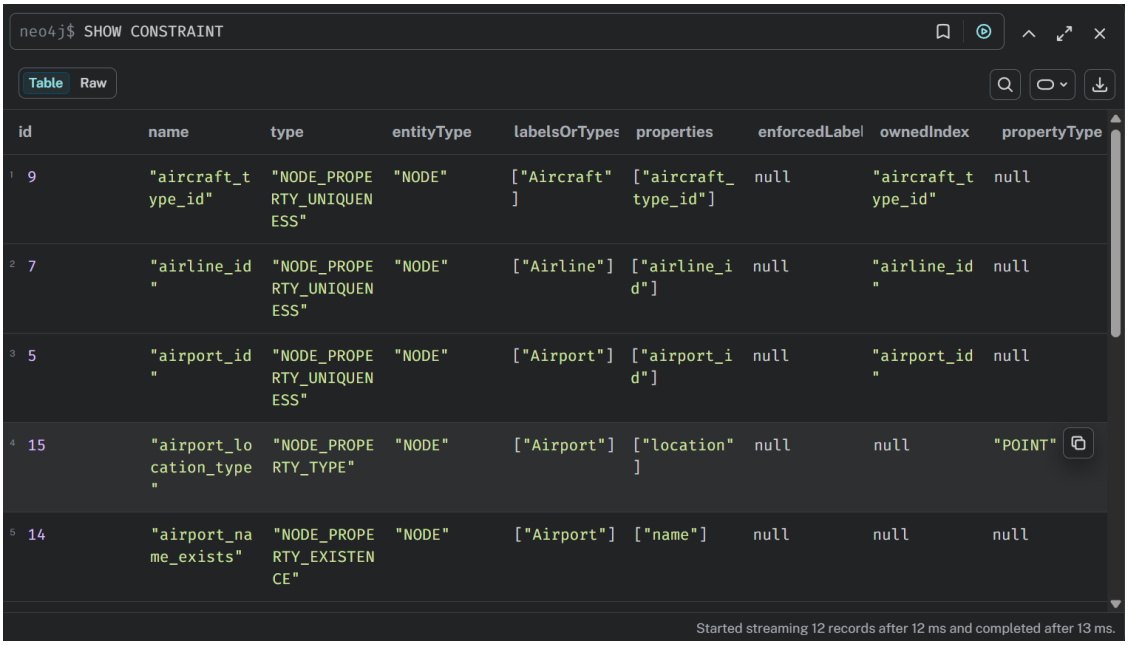


Figure 30: Constraints in Neo4j Browser (`SHOW CONSTRAINTS`)



Figure 31: Indexes in Neo4j Browser (`SHOW INDEXES`)

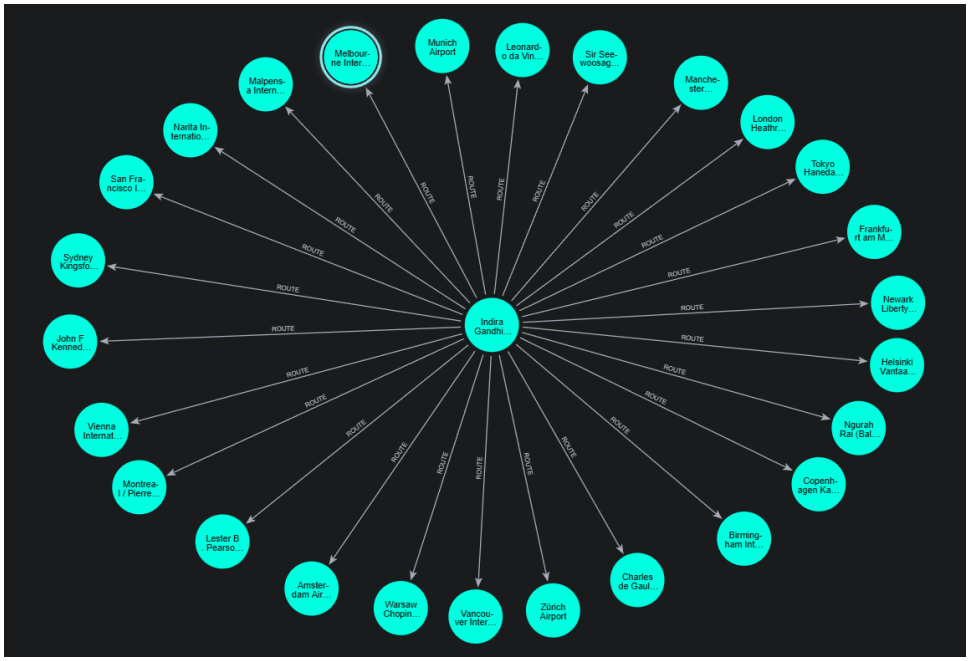


Figure 32: Neo4j Browser: Delhi's long-haul route network

Appendix G: User Manual

G.1 Starting the system

1. Open Neo4j Desktop and start the DBMS "DBMS Airlines".
2. In the project folder run `.venv/Scripts/python -m uvicorn app:app` and open http://localhost:8000 in a browser.
3. Pages are listed in the top bar as gates 01-06; on a phone, open them with the menu button.

G.2 Route Finder (Gate 01)

1. Type a city, airport name or code in From and To, and choose an airport from the list (or type a 3-letter code). The plane button swaps them.
2. Choose the number of stops (nonstop, up to 1, up to 2) and the extra time allowed for a greener trip (Auto or a fixed limit).
3. Press Find routes. The summary card shows how much CO2 the greenest option saves and the CO2 range of all bookable itineraries; each option appears as a boarding pass with its journey time, CO2 per seat, distance and flights, and a label such as "-13% CO2 vs typical".
4. Click a pass to highlight it on the map; open Legs for per-flight details. The unconstrained optimum at the bottom shows what a pure graph algorithm would choose.

G.3 Live Traffic (Gate 02)

1. Live now shows every simulated flight in the air at the current time; zoom in to see aircraft icons. Type an airline code or click a chip to highlight one airline.
2. Click an aircraft to open its boarding pass (route, progress, remaining time, CO2 per seat).
3. Replay 2015 plays the real flights of a chosen January 2015 day; use Play/Pause, the speed menu and the time slider. Colours show on time, delayed and diverted flights; a cross marks a cancellation.

G.4 Graph Explorer (Gate 03)

1. Choose Airport network, Flight or Disruption event, select the item and press Show.
2. Drag, zoom and click nodes or relationships to see their properties; double-click an airport to add its routes.
3. Switch to Map view to see the same airports and routes geographically.

G.5 Disruption Simulator (Gate 04, administrator)

1. Log in with Admin login (top right).
2. Add one or more airports, or choose a region; choose the cause and severity; optionally add a description.
3. Choose a route to watch, then press Simulate closure. After a few seconds the before/after table, the map and the impact report appear.
4. To undo, press Restore on the event in the Events list; Delete removes a manually created event.

G.6 Analytics (Gate 05)

Each card runs one query live. Change a parameter (for example the source airport) and press Run; open Cypher to see the query.

G.7 Manage Data (Gate 06, administrator)

1. Choose a tab: Airports, Airlines, Flights, Routes, Hubs or Events.
2. Search on the left; use Edit or Delete in a row; create records with the form on the right.
3. Airports are closed and reopened rather than deleted; an airport with routes cannot be deleted.
4. Flights: record an actual departure (the delay and status are calculated), change the status or reassign the aircraft.

G.8 Troubleshooting

Message or problem	Cause and fix
"Neo4j is not running"	Start the DBMS in Neo4j Desktop, then reload the page
"Admin login is not configured"	Add ADMIN_USER and ADMIN_PASSWORD to `.env` and restart the server
"XYZ is CLOSED: restore it in the Disruption Simulator first"	The airport is closed by a simulation; restore the event
Map background missing	No internet connection for map tiles; routes and aircraft are still drawn